

JAVA 17 OCP · YDS STUDY SERIES

Unit 08 · Lambdas and Functional Interfaces · Bilingual Notes

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu ana kaynak, [OCP_Java_SE17_Chapter1den_Itibaren.pdf](#) dosyasındaki Chapter 8 sayfaları 419–462'yi kaynak sırasını bozmadan kapsar. Tekrarlanan running header/footer ve basılı sayfa numaraları içerik sayılmamış; başlıklar, paragraflar, listeler, tablolar, figure/caption metinleri, kodlar, Summary, Exam Essentials ve Review Questions korunmuştur.

[Vocabulary](#) · [Grammar notes](#) · [Teknik hafıza notu](#)

Kaynak ve kapsam özeti

- Kaynak: [exam_lecture/OCP_Java_SE17_Chapter1den_Itibaren.pdf](#)
- Chapter: 8 · Lambdas and Functional Interfaces
- PDF sayfaları: 419–462
- Beklenen kaynak sayfa sayısı: 44
- Resmî Review Questions cevap eki: Appendix PDF sayfaları 936–939
- Beklenen cevap kaynağı sayfa sayısı: 4
- Eşleme biçimi: English paragraf → hemen altında Türkçe çeviri → varsa kod

İçindekiler

- 1 [Writing Simple Lambdas](#)
- 2 [Coding Functional Interfaces](#)
- 3 [Using Method References](#)
- 4 [Working with Built-in Functional Interfaces](#)
- 5 [Working with Variables in Lambdas](#)
- 6 [Summary / Özet](#)
- 7 [Exam Essentials / Sınav İçin Temel Noktalar](#)
- 8 [Review Questions / Gözden Geçirme Soruları](#)
- 9 [Official Review Question Answers / Resmî Cevaplar](#)

Chapter 8 · Lambdas and Functional Interfaces · Eksiksiz çift dilli kaynak

Chapter 8 · Lambdas and Functional Interfaces

English: OCP exam objectives covered in this chapter: Utilizing Java Object-Oriented Approach. Understand variable scopes, use local variable type inference, apply encapsulation, and make objects immutable. Create and use interfaces, identify functional interfaces, and utilize private, static, and default interface methods.

Türkçe: Bu bölümde kapsanan OCP sınav hedefleri: Java'nın Nesne Yönelimli Yaklaşımını kullanmak. Variable scope'ları anlamak, local variable type inference kullanmak, encapsulation uygulamak ve immutable nesneler oluşturmak. Interface'ler oluşturup kullanmak, functional interface'leri belirlemek ve private, static ve default interface method'larından yararlanmak.

English: In this chapter, we start by introducing lambdas, a new piece of syntax. Lambdas allow you to specify code that will be run later in the program.

Türkçe: Bu bölümde yeni bir syntax parçası olan lambda'ları tanıtarak başlıyoruz. Lambda'lar programın daha sonraki bir anında çalıştırılacak kodu belirtmenize olanak tanır.

English: Next, we introduce the concept of functional interfaces, showing how to write your own and identify whether an interface is a functional interface. After that, we introduce another new piece of syntax: method references. These are like a shorter form of lambdas.

Türkçe: Ardından functional interface kavramını tanıtıyor; kendi functional interface'inizi nasıl yazacağınızı ve bir interface'in functional interface olup olmadığını nasıl belirleyeceğinizi gösteriyoruz. Sonrasında yeni bir syntax parçasını daha, method reference'ları, ele alıyoruz. Bunlar lambda'ların daha kısa bir biçimi gibidir.

English: Then we introduce the functional interfaces you need to know for the exam. Finally, we emphasize how variables fit into lambdas.

Türkçe: Daha sonra sınav için bilmeniz gereken functional interface'leri tanıtıyoruz. Son olarak variable'ların lambda'larla nasıl bir araya geldiğini vurguluyoruz.

English: Lambdas, method references, and functional interfaces are used quite a bit in Chapter 9, “Collections and Generics” and Chapter 10, “Streams.”

Türkçe: Lambda'lar, method reference'lar ve functional interface'ler Chapter 9, “Collections and Generics” ile Chapter 10, “Streams” bölümlerinde oldukça sık kullanılır.

Writing Simple Lambdas

English: Java is an object-oriented language at heart. You've seen plenty of objects by now. Functional programming is a way of writing code more declaratively. You specify what you want to do rather than dealing with the state of objects. You focus more on expressions than loops.

Türkçe: Java özünde object-oriented bir dildir. Şimdiye kadar çok sayıda object gördünüz. Functional programming, kodu daha declarative biçimde yazma yoludur. Object'lerin state'iyile uğraşmak yerine ne yapmak istediğinizi belirtirsiniz. Loop'lardan çok expression'lara odaklanırsınız.

English: Functional programming uses lambda expressions to write code. A lambda expression is a block of code that gets passed around. You can think of a lambda expression as an unnamed method existing inside an anonymous class like the ones you saw in Chapter 7, “Beyond Classes.” It has parameters and a body just like full-fledged methods do, but it doesn't have a name like a real method. Lambda expressions are often referred to as lambdas for short. You might also know them as closures if Java isn't your first language. If you had a bad experience with closures in the past, don't worry. They are far simpler in Java.

Türkçe: Functional programming, kod yazmak için lambda expression'ları kullanır. Lambda expression, bir yerden başka bir yere geçirilen kod bloğudur. Lambda expression'ı Chapter 7, “Beyond Classes” bölümünde gördüğünüz anonymous class'ların içinde var olan adsız bir method gibi düşünebilirsiniz. Tam teşekküllü method'lar gibi parameter'ları ve bir body'si vardır; ancak gerçek bir method gibi adı yoktur. Lambda expression'lardan kısaca lambda diye söz edilir. Java ilk diliniz değilse bunları closure adıyla da biliyor olabilirsiniz. Geçmişte closure'larla kötü bir deneyim yaşadıysanız endişelenmeyin; Java'da çok daha basittirler.

English: Lambdas allow you to write powerful code in Java. In this section, we cover an example of why lambdas are helpful and the syntax of lambdas.

Türkçe: Lambda'lar Java'da güçlü kod yazmanıza olanak tanır. Bu bölümde lambda'ların neden yararlı olduğunu gösteren bir örneği ve lambda syntax'ını ele alıyoruz.

Looking at a Lambda Example

English: Our goal is to print out all the animals in a list according to some criteria. We show you how to do this without lambdas to illustrate how lambdas are useful. We start with the `Animal` record:

Türkçe: Amacımız bir listedeki bütün hayvanları bazı ölçütlere göre yazdırmaktır. Lambda'ların nasıl yararlı olduğunu göstermek için önce bunu lambda kullanmadan nasıl yapacağınızı gösteriyoruz. `Animal` record'u ile başlıyoruz:

```
public record Animal(String species, boolean canHop, boolean canSwim) { }
```

English: The `Animal` record has three fields. Let's say we have a list of animals, and we want to process the data based on a particular attribute. For example, we want to print all animals that can hop. We can define an interface to generalize this concept and support a large variety of checks:

Türkçe: `Animal` record'unun üç field'ı vardır. Bir hayvan listemiz bulunduğunu ve veriyi belirli bir attribute'a göre işlemek istediğimizi varsayalım. Örneğin zıplayabilen bütün hayvanları yazdırmak istiyoruz. Bu kavramı genelleştirmek ve çok çeşitli kontrolleri desteklemek için bir interface tanımlayabiliriz:

```
public interface CheckTrait {  
    boolean test(Animal a);  
}
```

English: The first thing we want to check is whether the `Animal` can hop. We provide a class that implements our interface:

Türkçe: Kontrol etmek istediğimiz ilk şey `Animal` nesnesinin zıplayıp zıplayamadığıdır. Interface'imizi implement eden bir class sağlıyoruz:

```
public class CheckIfHopper implements CheckTrait {  
    public boolean test(Animal a) {  
        return a.canHop();  
    }  
}
```

English: This class may seem simple—and it is. This is part of the problem that lambdas solve. Just bear with us for a bit. Now we have everything we need to write our code to find out if an `Animal` can hop:

Türkçe: Bu class basit görünebilir—gerçekten de basittir. Bu durum, lambda'ların çözdüğü problemin bir parçasıdır. Biraz daha sabredin. Artık bir `Animal` nesnesinin zıplayıp zıplayamadığını öğrenen kodu yazmak için gereken her şeye sahibiz:

```
1: import java.util.*;
2: public class TraditionalSearch {
3:     public static void main(String[] args) {
4:
5:         // list of animals
6:         var animals = new ArrayList<Animal>();
7:         animals.add(new Animal("fish", false, true));
8:         animals.add(new Animal("kangaroo", true, false));
9:         animals.add(new Animal("rabbit", true, false));
10:        animals.add(new Animal("turtle", false, true));
11:
12:        // pass class that does check
13:        print(animals, new CheckIfHopper());
14:    }
15:    private static void print(List<Animal> animals, CheckTrait checker) {
16:        for (Animal animal : animals) {
17:
18:            // General check
19:            if (checker.test(animal))
20:                System.out.print(animal + " ");
21:        }
22:        System.out.println();
23:    }
24: }
```

English: Line 6 shows configuring an `ArrayList` with a specific type of `Animal`. The `print()` method on line 15 is very general—it can check for any trait. This is good design. It shouldn't need to know what specifically we are searching for in order to print a list of animals.

Türkçe: Satır 6, bir `ArrayList`'in belirli bir `Animal` type'ıyla yapılandırılmasını gösterir. Satır 15'teki `print()` method'u oldukça geneldir; herhangi bir trait'i kontrol edebilir. Bu iyi bir tasarımdır. Hayvan listesini yazdırmak için özellikle ne aradığımızı bilmesi gerekmez.

English: What happens if we want to print the `Animals` that swim? Sigh. We need to write another class, `CheckIfSwims`. Granted, it is only a few lines, but it is a whole new file. Then we need to add a new line under line 13 that instantiates that class. That's two things just to do another check.

Türkçe: Ya yüzebilen `Animal` nesnelerini yazdırmak istersek ne olur? Maalesef başka bir class, `CheckIfSwims`, yazmamız gerekir. Yalnız birkaç satırdan oluştuğu doğru olsa da bu tamamen yeni bir dosyadır. Ardından satır 13'ün altına bu class'ı instantiate eden yeni bir satır eklemeliyiz. Yalnızca başka bir kontrol yapmak için iki ayrı işlem gerekir.

English: Why can't we specify the logic we care about right here? It turns out that we can, with lambda expressions. We could repeat the whole class here and make you find the one line that changed. Instead, we just show you that we can keep our `print()` method declaration unchanged. Let's replace line 13 with the following, which uses a lambda:

Türkçe: İlgilendiğimiz logic'i neden tam burada belirtmeyelim? Lambda expression'larla bunu yapabildiğimiz ortaya çıkıyor. Bütün class'ı burada tekrarlayıp değişen tek satırı size buldurabilirdik. Bunun yerine `print()` method declaration'ını değiştirmeden koruyabildiğimizi gösteriyoruz. Satır 13'ü lambda kullanan şu satırla değiştirelim:

```
13: print(animals, a -> a.canHop());
```

English: Don't worry that the syntax looks a little funky. You'll get used to it, and we describe it in the next section. We also explain the bits that look like magic. For now, just focus on how easy it is to read. We are telling Java that we only care if an `Animal` can hop.

Türkçe: Syntax'ın biraz tuhaf görünmesine aldırmayın. Buna alışacaksınız; ayrıca sonraki bölümde açıklıyoruz. Sihir gibi görünen parçaları da ele alıyoruz. Şimdilik yalnızca okunmasının ne kadar kolay olduğuna odaklanın. Java'ya yalnızca bir `Animal` nesnesinin zıplayabilmesiyle ilgilendiğimizi söylüyoruz.

English: It doesn't take much imagination to figure out how we would add logic to get the `Animals` that can swim. We only have to add one line of code—no need for an extra class to do something simple. Here's that other line:

Türkçe: Yüzebilen `Animal` nesnelerini elde edecek logic'i nasıl ekleyeceğimizi anlamak zor değildir. Yalnızca tek bir kod satırı eklememiz gerekir; basit bir şey yapmak için fazladan class'a ihtiyaç yoktur. İşte o diğer satır:

```
13: print(animals, a -> a.canSwim());
```

English: How about `Animals` that cannot swim?

Türkçe: Peki yüzemeyen `Animal` nesneleri?

```
13: print(animals, a -> !a.canSwim());
```

English: The point is that it is really easy to write code that uses lambdas once you get the basics in place. This code uses a concept called deferred execution. Deferred execution means that code is specified now but will run later. In this case, "later" is inside the `print()` method body, as opposed to when it is passed to the method.

Türkçe: Temelleri yerleştirdikten sonra lambda kullanan kod yazmak gerçekten kolaydır. Bu kod deferred execution (ertelenmiş yürütme) denen bir kavramı kullanır. Deferred execution, kodun şimdi belirtilip daha sonra çalıştırılması demektir. Bu örnekte "daha sonra", kodun method'a geçirildiği anın aksine `print()` method body'sinin içidir.

Learning Lambda Syntax

English: One of the simplest lambda expressions you can write is the one you just saw:

Türkçe: Yazabileceğiniz en basit lambda expression'lardan biri az önce gördüğünüzdür:

```
a -> a.canHop()
```

English: Lambdas work with interfaces that have exactly one abstract method. In this case, Java looks at the `CheckTrait` interface, which has one method. The lambda in our example suggests that Java should call a method with an `Animal` parameter that returns a `boolean` value that's the result of `a.canHop()`. We know all this because we wrote the code. But how does Java know?

Türkçe: Lambda'lar tam olarak bir abstract method'u bulunan interface'lerle çalışır. Bu durumda Java, bir method'u olan `CheckTrait` interface'ine bakar. Örneğimizdeki lambda, Java'nın bir `Animal` parameter'ı alan ve `a.canHop()` sonucundaki `boolean` değeri döndüren bir method çağırması gerektiğini ifade eder. Kodu biz yazdığımız için bunların hepsini biliyoruz. Peki Java nasıl biliyor?

English: Java relies on context when figuring out what lambda expressions mean. Context refers to where and how the lambda is interpreted. For example, if we see someone in line to enter the zoo and they have their wallet out, it is fair to assume they want to buy zoo tickets.

Türkçe: Java, lambda expression'ların ne anlama geldiğini çözerken context'e dayanır. Context, lambda'nın nerede ve nasıl yorumlandığını ifade eder. Örneğin hayvanat bahçesine giriş kuyruğunda cüzdanını çıkarmış birini görürsek bilet almak istediğini varsaymak mantıklıdır.

English: Alternatively, if they are in the concession line with their wallet out, they are probably hungry.

Türkçe: Buna karşılık cüzdanları ellerindeyken yiyecek satış kuyruğunda bulunuyorlarsa muhtemelen açtırlar.

English: Referring to our earlier example, we passed the lambda as the second parameter of the `print()` method:

Türkçe: Önceki örneğimize dönersek lambda'yı `print()` method'unun ikinci parameter'ı olarak geçirdik:

```
print(animals, a -> a.canHop());
```

English: The `print()` method expects a `CheckTrait` as the second parameter:

Türkçe: `print()` method'u ikinci parameter olarak bir `CheckTrait` bekler:

```
private static void print(List<Animal> animals, CheckTrait checker) { ... }
```

English: Since we are passing a lambda instead, Java tries to map our lambda to the abstract method declaration in the `CheckTrait` interface:

Türkçe: Bunun yerine bir lambda geçirdiğimiz için Java lambda'mızı `CheckTrait` interface'indeki abstract method declaration'a eşlemeye çalışır:

```
boolean test(Animal a);
```

English: Since that interface's method takes an `Animal`, the lambda parameter has to be an `Animal`. And since that interface's method returns a `boolean`, we know the lambda returns a `boolean`.

Türkçe: Interface'in method'u bir `Animal` aldığı için lambda parameter'ı da bir `Animal` olmak zorundadır. Aynı method `boolean` döndürdüğü için lambda'nın da `boolean` döndürdüğünü biliriz.

English: The syntax of lambdas is tricky because many parts are optional. These two lines do the exact same thing:

Türkçe: Birçok parça optional olduğundan lambda syntax'ı yanıltıcı olabilir. Şu iki satır tamamen aynı işi yapar:

```
a -> a.canHop()  
(Animal a) -> { return a.canHop(); }
```

English: Let's look at what is going on here. The first example, shown in Figure 8.1, has three parts: A single parameter specified with the name `a`; the arrow operator (`->`) to separate the parameter and body; and a body that calls a single method and returns the result of that method.

Türkçe: Burada neler olduğuna bakalım. Figure 8.1'de gösterilen ilk örnek üç parçadan oluşur: `a` adıyla belirtilen tek parameter; parameter ile body'yi ayıran arrow operator (`->`); tek bir method çağıran ve bu method'un sonucunu döndüren body.

Figure 8.1 · Lambda syntax omitting optional parts

English — figure callouts: Parameter name; arrow; body. **Türkçe — şekil çağrılar:** Parameter adı; arrow; body.

```
a -> a.canHop()
```

English: The second example shows the most verbose form of a lambda that returns a `boolean` (see Figure 8.2): A single parameter specified with the name `a` and stating that the type is `Animal`; the arrow operator (`->`) to separate the parameter and body; and a body that has one or more lines of code, including a semicolon and a `return` statement.

Türkçe: İkinci örnek, `boolean` döndüren bir lambda'nın en ayrıntılı biçimini gösterir (Figure 8.2): `a` adıyla belirtilen ve type'ının `Animal` olduğu açıkça yazılan tek parameter; parameter ile body'yi ayıran arrow operator (`->`); semicolon ve `return` statement dahil bir veya daha fazla kod satırı içeren body.

Figure 8.2 · Lambda syntax including optional parts

English — figure callouts: Parameter name; arrow; body. **Türkçe — şekil çağrılar:** Parameter adı; arrow; body.

```
(Animal a) -> { return a.canHop(); }
```

English: The parentheses around the lambda parameters can be omitted only if there is a single parameter and its type is not explicitly stated. Java does this because developers commonly use lambda expressions this way and can do as little typing as possible.

Türkçe: Lambda parameter'larını çevreleyen parentheses yalnızca tek bir parameter varsa ve type'ı açıkça belirtilmemişse atlanabilir. Java bunu, geliştiriciler lambda expression'ları çoğunlukla bu şekilde kullandığı ve mümkün olduğunca az yazabilsin diye yapar.

English: It shouldn't be news to you that we can omit braces when we have only a single statement. We did this with `if` statements and loops already. Java allows you to omit a `return` statement and semicolon (`;`) when no braces are used. This special shortcut doesn't work when you have two or more statements. At least this is consistent with using `{ }` to create blocks of code elsewhere.

Türkçe: Yalnızca tek statement olduğunda braces'i atlayabilmemiz sizin için yeni olmamalıdır. Bunu `if` statement'ları ve loop'larda zaten yaptık. Java, braces kullanılmadığında `return` statement'ını ve semicolon'u (`;`) atlamanıza izin verir. Bu özel kısayol iki veya daha fazla statement bulunduğu çalışmaz. En azından bu, başka yerlerde kod blokları oluşturmak için `{ }` kullanılmasıyla tutarlıdır.

English: The syntax in Figure 8.1 and Figure 8.2 can be mixed and matched. For example, the following are valid:

Türkçe: Figure 8.1 ve Figure 8.2'deki syntax parçaları farklı şekillerde birleştirilebilir. Örneğin aşağıdakiler geçerlidir:

```
a -> { return a.canHop(); }
(Animal a) -> a.canHop()
```

English: Here's a fun fact: `s -> {}` is a valid lambda. If there is no code on the right side of the expression, you don't need the semicolon or `return` statement.

Türkçe: İlginç bir bilgi: `s -> {}` geçerli bir lambda'dır. Expression'ın sağ tarafında kod yoksa semicolon veya `return` statement gerekmez.

English: Table 8.1 shows examples of valid lambdas that return a `boolean`.

Türkçe: Table 8.1, `boolean` döndüren geçerli lambda örneklerini gösterir.

Table 8.1 · Valid lambdas that return a boolean

Lambda / Lambda ifadesi	Number of parameters / Parameter sayısı
<code>() -> true</code>	0
<code>x -> x.startsWith("test")</code>	1
<code>(String x) -> x.startsWith("test")</code>	1
<code>(x, y) -> { return x.startsWith("test"); }</code>	2
<code>(String x, String y) -> x.startsWith("test")</code>	2

English: The first row takes zero parameters and always returns the `boolean` value `true`. The second row takes one parameter and calls a method on it, returning the result. The third row does the same, except that it explicitly defines the type of the variable. The final two rows take two parameters and ignore one of them—there isn't a rule that says you must use all defined parameters.

Türkçe: İlk satır sıfır parameter alır ve her zaman `boolean` değer `true` döndürür. İkinci satır bir parameter alır, üzerinde bir method çağırır ve sonucu döndürür. Üçüncü satır da aynısını yapar; tek fark variable'ın type'ını açıkça tanımlamasıdır. Son iki satır iki parameter alıp bunlardan birini görmezden gelir; tanımlanmış bütün parameter'ları kullanmak zorunda olduğunuzu söyleyen bir kural yoktur.

English: Now let's make sure you can identify invalid syntax for each row in Table 8.2, where each lambda is supposed to return a `boolean`. Make sure you understand what's wrong with these.

Türkçe: Şimdi her lambda'nın `boolean` döndürmesi gereken Table 8.2'deki her satır için invalid syntax'ı belirleyebildiğinizden emin olalım. Bunların neden yanlış olduğunu anladığınızdan emin olun.

Table 8.2 · Invalid lambdas that should return a boolean

Invalid lambda / Geçersiz lambda	Reason / Neden
<code>x, y -> x.startsWith("fish")</code>	Missing parentheses on left / Solda parentheses eksik
<code>x -> { x.startsWith("camel"); }</code>	Missing return on right / Sağda <code>return</code> eksik
<code>x -> { return x.startsWith("giraffe") }</code>	Missing semicolon inside braces / Braces içinde semicolon eksik
<code>String x -> x.endsWith("eagle")</code>	Missing parentheses on left / Solda parentheses eksik

English: Remember that the parentheses are optional only when there is one parameter and it doesn't have a type declared. Those are the basics of writing a lambda. At the end of the chapter, we cover additional rules about using variables in a lambda.

Türkçe: Parentheses'in yalnızca bir parameter bulunduğu ve bu parameter'ın type'ı bildirilmediğinde optional olduğunu unutmayın. Bunlar lambda yazmanın temel kurallarıdır. Bölümün sonunda lambda içinde variable kullanmaya ilişkin ek kuralları ele alıyoruz.

Assigning Lambdas to `var`

English: Why do you think this line of code doesn't compile?

Türkçe: Sizce bu kod satırı neden derlenmiyor?

```
var invalid = (Animal a) -> a.canHop(); // DOES NOT COMPILE
```

English: Remember when we talked about Java inferring information about the lambda from the context? Well, `var` assumes the type based on the context as well. There's not enough context here! Neither the lambda nor `var` have enough information to determine what type of functional interface should be used.

Türkçe: Java'nın lambda hakkındaki bilgiyi context'ten infer etmesini konuştuğumuzu hatırlıyor musunuz? `var` da type'ı context'e göre varsayar. Burada yeterli context yoktur! Ne lambda ne de `var`, hangi functional interface type'ının kullanılması gerektiğini belirlemek için yeterli bilgiye sahiptir.

Coding Functional Interfaces

English: Earlier in the chapter, we declared the `CheckTrait` interface, which has exactly one method for implementers to write. Lambdas have a special relationship with such interfaces. In fact, these interfaces have a name. A functional interface is an interface that contains a single abstract method. Your friend Sam can help you remember this because it is officially known as a single abstract method (SAM) rule.

Türkçe: Bölümün önceki kısmında implement edenlerin yazacağı tam olarak bir method'u bulunan `CheckTrait` interface'ini bildirdik. Lambda'ların bu tür interface'lerle özel bir ilişkisi vardır. Aslında bu interface'lerin bir adı vardır. Functional interface, tek bir abstract method içeren interface'tir. Arkadaşınız Sam bunu hatırlamanıza yardım edebilir; çünkü bu kuralın resmî adı single abstract method (SAM) kuralıdır.

Defining a Functional Interface

English: Let's take a look at an example of a functional interface and a class that implements it:

Türkçe: Bir functional interface ve onu implement eden class örneğine bakalım:

```
@FunctionalInterface
public interface Sprint {
    public void sprint(int speed);
}

public class Tiger implements Sprint {
    public void sprint(int speed) {
        System.out.println("Animal is sprinting fast! " + speed);
    }
}
```

English: In this example, the `Sprint` interface is a functional interface because it contains exactly one abstract method, and the `Tiger` class is a valid class that implements the interface.

Türkçe: Bu örnekte `Sprint` interface'i tam olarak bir abstract method içerdiği için functional interface'tir; `Tiger` class'ı ise bu interface'i implement eden geçerli bir class'tır.

The `@FunctionalInterface` Annotation

English: The `@FunctionalInterface` annotation tells the compiler that you intend for the code to be a functional interface. If the interface does not follow the rules for a functional interface, the compiler will give you an error.

Türkçe: `@FunctionalInterface` annotation'ı compiler'a kodun functional interface olmasını amaçladığınızı bildirir. Interface, functional interface kurallarına uymuyorsa compiler hata verir.

```
@FunctionalInterface
public interface Dance { // DOES NOT COMPILE
    void move();
    void rest();
}
```

English: Java includes `@FunctionalInterface` on some, but not all, functional interfaces. This annotation means the authors of the interface promise it will be safe to use in a lambda in the future. However, just because you don't see the annotation doesn't mean it's not a functional interface. Remember that having exactly one abstract method is what makes it a functional interface, not the annotation.

Türkçe: Java, functional interface'lerin bazılarında `@FunctionalInterface` kullanır; fakat hepsinde kullanmaz. Bu annotation, interface'in yazarlarının onun gelecekte lambda ile güvenle kullanılabileceği sözünü vermesi anlamına gelir. Bununla birlikte annotation'ı görmemeniz, onun functional interface olmadığı anlamına gelmez. Bir interface'i functional yapan şeyin annotation değil, tam olarak bir abstract method'a sahip olması olduğunu unutmayın.

English: Consider the following four interfaces. Given our previous `Sprint` functional interface, which of the following are functional interfaces?

Türkçe: Aşağıdaki dört interface'i düşünün. Önceki `Sprint` functional interface'imizi göz önünde bulundurduğumuzda bunlardan hangileri functional interface'tir?

```
public interface Dash extends Sprint {}

public interface Skip extends Sprint {
    void skip();
}

public interface Sleep {
    private void snore() {}
    default int getZzz() { return 1; }
}

public interface Climb {
    void reach();
    default void fall() {}
    static int getBackUp() { return 100; }
    private static boolean checkHeight() { return true; }
}
```

English: All four of these are valid interfaces, but not all of them are functional interfaces. The `Dash` interface is a functional interface because it extends the `Sprint` interface and inherits the single abstract method `sprint()`. The `Skip` interface is not a valid functional interface because it has two abstract methods: the inherited `sprint()` method and the declared `skip()` method.

Türkçe: Bunların dördü de geçerli interface'tir; ancak hepsi functional interface değildir. `Dash`, `Sprint` interface'ini extend edip tek abstract method `sprint()` i inherit ettiği için functional interface'tir. `Skip` ise iki abstract method'a—inherit edilen `sprint()` ile bildirilen `skip()` a— sahip olduğundan geçerli bir functional interface değildir.

English: The `Sleep` interface is also not a valid functional interface. Neither `snore()` nor `getZzz()` meets the criteria of a single abstract method. Even though default methods function like abstract methods, in that they can be overridden in a class implementing the interface, they are insufficient for satisfying the single abstract method requirement.

Türkçe: `Sleep` interface'i de geçerli bir functional interface değildir. Ne `snore()` ne de `getZzz()` single abstract method ölçütünü karşılar. Default method'lar interface'i implement eden class'ta override edilebilmeleri bakımından abstract method'lar gibi işlev görse de single abstract method gereksinimini karşılamak için yeterli değildir.

English: Finally, the `Climb` interface is a functional interface. Despite defining a slew of methods, it contains only one abstract method: `reach()`.

Türkçe: Son olarak `Climb` interface'i functional interface'tir. Çok sayıda method tanımlamasına karşın yalnızca bir abstract method içerir: `reach()`.

Adding Object Methods

English: All classes inherit certain methods from `Object`. For the exam, you should know the following `Object` method signatures:

Türkçe: Bütün class'lar `Object`ten belirli method'ları inherit eder. Sınav için aşağıdaki `Object` method signature'larını bilmelisiniz:

```
public String toString()
public boolean equals(Object)
public int hashCode()
```

English: We bring this up now because there is one exception to the single abstract method rule that you should be familiar with. If a functional interface includes an abstract method with the same signature as a public method found in `Object`, those methods do not count toward the single abstract method test.

Türkçe: Bunu şimdi ele almamızın nedeni, single abstract method kuralının bilmeniz gereken bir istisnasının bulunmasıdır. Bir functional interface, `Object`teki public bir method'la aynı signature'a sahip abstract method içeriyorsa bu method'lar single abstract method testinde sayılmaz.

English: The motivation behind this rule is that any class that implements the interface will inherit from `Object`, as all classes do, and therefore always implement these methods.

Türkçe: Bu kuralın gerekçesi, interface'i implement eden herhangi bir class'ın bütün class'lar gibi `Object`ten inherit edecek ve dolayısıyla bu method'ları her zaman implement edecek olmasıdır.

English: Since Java assumes all classes extend from `Object`, you also cannot declare an interface method that is incompatible with `Object`. For example, declaring an abstract method `int toString()` in an interface would not compile since `Object`'s version of the method returns a `String`.

Türkçe: Java bütün class'ların `Object`ten extend edildiğinden varsaydığından `Object` ile incompatible bir interface method'u da bildiremezsiniz. Örneğin bir interface'te `int toString()` abstract method'unu bildirmek derlenmez; çünkü method'un `Object`teki sürümü `String` döndürür.

English: Let's take a look at an example. Is the `Soar` interface a functional interface?

Türkçe: Bir örneğe bakalım. `Soar` interface'i bir functional interface midir?

Editor note: The source calls `Soar` a “class,” although the declaration uses `interface`; the technically correct term is used above. **Editör notu:** Kaynak, declaration'da `interface` kullanılmasına rağmen `Soar`'ı “class” diye adlandırır; yukarıda teknik olarak doğru terim kullanılmıştır.

```
public interface Soar {
    abstract String toString();
}
```

English: It is not. Since `toString()` is a public method implemented in `Object`, it does not count toward the single abstract method test. On the other hand, the following implementation of `Dive` is a functional interface:

Türkçe: Değildir. `toString()`, `Object`te implement edilen public bir method olduğundan single abstract method testinde sayılmaz. Buna karşılık aşağıdaki `Dive` implementation'ı functional interface'tir:

```
public interface Dive {
    String toString();
    public boolean equals(Object o);
    public abstract int hashCode();
    public void dive();
}
```

English: The `dive()` method is the single abstract method, while the others are not counted since they are public methods defined in the `Object` class.

Türkçe: `dive()` single abstract method'dur; diğerleri `Object` class'ında tanımlanmış public method'lar olduğundan sayılmaz.

English: Be wary of examples that resemble methods in the `Object` class but are not actually defined in the `Object` class. Do you see why the following is not a valid functional interface?

Türkçe: `Object` class'ındaki method'lara benzeyen ancak gerçekte `Object`'te tanımlanmamış örneklerle karşı dikkatli olun. Aşağıdakinin neden geçerli bir functional interface olmadığını görüyor musunuz?

```
public interface Hibernate {
    String toString();
    public boolean equals(Hibernate o);
    public abstract int hashCode();
    public void rest();
}
```

English: Despite looking a lot like our `Dive` interface, the `Hibernate` interface uses `equals(Hibernate)` instead of `equals(Object)`. Because this does not match the method signature of the `equals(Object)` method defined in the `Object` class, this interface is counted as containing two abstract methods: `equals(Hibernate)` and `rest()`.

Türkçe: `Dive` interface'imize çok benzemesine rağmen `Hibernate`, `equals(Object)` yerine `equals(Hibernate)` kullanır. Bu, `Object` class'ında tanımlanan `equals(Object)` method'unun signature'ıyla eşleşmediğinden bu interface'in iki abstract method içerdiği kabul edilir: `equals(Hibernate)` ve `rest()`.

Using Method References

English: Method references are another way to make the code easier to read, such as simply mentioning the name of the method. Like lambdas, it takes time to get used to the new syntax. In this section, we show the syntax along with the four types of method references. We also mix in lambdas with method references.

Türkçe: Method reference'lar, örneğin yalnızca method'un adını belirterek kodun daha kolay okunmasını sağlamanın başka bir yoludur. Lambda'larda olduğu gibi yeni syntax'a alışmak zaman alır. Bu bölümde syntax'ı dört method reference türüyle birlikte gösteriyoruz. Lambda'larla method reference'ları birlikte de kullanıyoruz.

English: Suppose we are coding a duckling that is trying to learn how to quack. First we have a functional interface:

Türkçe: Vaklamayı öğrenmeye çalışan bir ördek yavrusu kodladığımızı varsayalım. Önce bir functional interface'imiz var:

```
public interface LearnToSpeak {
    void speak(String sound);
}
```

English: Next, we discover that our duckling is lucky. There is a helper class that the duckling can work with. We've omitted the details of teaching the duckling how to quack and left the part that calls the functional interface:

Türkçe: Ardından ördek yavrumuzun şanslı olduğunu keşfediyoruz. Birlikte çalışabileceği bir helper class vardır. Ördek yavrusuna vaklamayı öğretmenin ayrıntılarını atlayıp functional interface'i çağıran kısmı bıraktık:

```
public class DuckHelper {
    public static void teacher(String name, LearnToSpeak trainer) {
        // Exercise patience (omitted)
        trainer.speak(name);
    }
}
```

English: Finally, it is time to put it all together and meet our little `Duckling`. This code implements the functional interface using a lambda:

Türkçe: Sonunda bütün parçaları bir araya getirip küçük `Duckling`imizle tanışma zamanı geldi. Bu kod functional interface'i lambda kullanarak implement eder:

```
public class Duckling {
    public static void makeSound(String sound) {
        LearnToSpeak learner = s -> System.out.println(s);
        DuckHelper.teacher(sound, learner);
    }
}
```

English: Not bad. There's a bit of redundancy, though. The lambda declares one parameter named `s`. However, it does nothing other than pass that parameter to another method. A method reference lets us remove that redundancy and instead write this:

Türkçe: Fena değil. Yine de biraz redundancy vardır. Lambda, `s` adlı bir parameter bildirir; ancak bu parameter'ı başka bir method'a geçirmekten başka hiçbir şey yapmaz. Method reference bu tekrarı kaldırıp yerine şunu yazmamıza olanak tanır:

```
LearnToSpeak learner = System.out::println;
```

English: The `::` operator tells Java to call the `println()` method later. It will take a little while to get used to the syntax. Once you do, you may find your code is shorter and less distracting without writing as many lambdas.

Türkçe: `::` operator, Java'ya `println()` method'unu daha sonra çağırmasını söyler. Syntax'a alışmak biraz zaman alacaktır. Alıştıңызda bu kadar çok lambda yazmadan kodunuzun daha kısa ve daha az dikkat dağıtıcı olduğunu görebilirsiniz.

English: Remember that `::` is like a lambda, and it is used for deferred execution with a functional interface. You can even imagine the method reference as a lambda if it helps you.

Türkçe: `::` ifadesinin lambda gibi olduğunu ve functional interface ile deferred execution için kullanıldığını unutmayın. Yardımcı oluyorsa method reference'ı bir lambda olarak bile düşünebilirsiniz.

English: A method reference and a lambda behave the same way at runtime. You can pretend the compiler turns your method references into lambdas for you.

Türkçe: Method reference ile lambda runtime'da aynı şekilde davranır. Compiler'ın method reference'lerinizi sizin için lambda'lara dönüştürdüğünü düşünebilirsiniz.

English: There are four formats for method references: static methods; instance methods on a particular object; instance methods on a parameter to be determined at runtime; and constructors.

Türkçe: Method reference'ların dört biçimi vardır: static method'lar; belirli bir object üzerindeki instance method'lar; runtime'da belirlenecek bir parameter üzerindeki instance method'lar ve constructor'lar.

English: Let's take a brief look at each of these in turn. In each example, we show the method reference and its lambda equivalent. For now, we create a separate functional interface for each example. In the next section, we introduce built-in functional interfaces so you don't have to keep writing your own.

Türkçe: Bunların her birine sırayla kısaca bakalım. Her örnekte method reference'ı ve lambda equivalent'ını gösteriyoruz. Şimdilik her örnek için ayrı bir functional interface oluşturuyoruz. Sonraki bölümde sürekli kendi interface'inizi yazmak zorunda kalmamanız için built-in functional interface'leri tanıtıyoruz.

Calling static Methods

English: For the first example, we use a functional interface that converts a `double` to a `long`:

Türkçe: İlk örnekte `double`ı `long`a dönüştüren bir functional interface kullanıyoruz:

```
interface Converter {  
    long round(double num);  
}
```

English: We can implement this interface with the `round()` method in `Math`. Here we assign a method reference and a lambda to this functional interface:

Türkçe: Bu interface'i `Math` içindeki `round()` method'uyla implement edebiliriz. Burada bu functional interface'e bir method reference ve bir lambda atıyoruz:

```
14: Converter methodRef = Math::round;  
15: Converter lambda = x -> Math.round(x);  
16:  
17: System.out.println(methodRef.round(100.1)); // 100
```

English: On line 14, we reference a method with one parameter, and Java knows that it's like a lambda with one parameter. Additionally, Java knows to pass that parameter to the method.

Türkçe: Satır 14'te bir parameter'lı olan method'a reference veriyoruz ve Java bunun bir parameter'lı lambda gibi olduğunu bilir. Ayrıca Java bu parameter'ı method'a geçirmesi gerektiğini bilir.

English: Wait a minute. You might be aware that the `round()` method is overloaded—it can take a `double` or a `float`. How does Java know that we want to call the version with a `double`? With both lambdas and method references, Java infers information from the context. In this case, we said that we were declaring a `Converter`, which has a method taking a `double` parameter. Java looks for a method that matches that description. If it can't find it or finds multiple matches, then the compiler will report an error. The latter is sometimes called an ambiguous type error.

Türkçe: Bir dakika. `round()` method'unun overloaded olduğunu; `double` veya `float` alabildiğini biliyor olabilirsiniz. Java `double` alan sürümü çağırmak istediğimizi nasıl bilir? Java, hem lambda'larda hem method reference'larda bilgiyi context'ten infer eder. Burada `double` parameter alan bir method'u bulunan `Converter` bildirdiğimizi söyledik. Java bu tanıma uyan bir method arar. Bulamazsa veya birden fazla eşleşme bulursa compiler hata bildirir. İkinci durum bazen ambiguous type error diye adlandırılır.

Calling Instance Methods on a Particular Object

English: For this example, our functional interface checks if a `String` starts with a specified value:

Türkçe: Bu örnekte functional interface'imiz bir `String`'in belirtilen değerle başlayıp başlamadığını kontrol eder:

```
interface StringStart {
    boolean beginningCheck(String prefix);
}
```

English: Conveniently, the `String` class has a `startsWith()` method that takes one parameter and returns a `boolean`. Let's look at how to use method references with this code:

Türkçe: Neyse ki `String` class'ının bir parameter alıp `boolean` döndüren `startsWith()` method'u vardır. Bu kodla method reference'ların nasıl kullanılacağına bakalım:

```
18: var str = "Zoo";
19: StringStart methodRef = str::startsWith;
20: StringStart lambda = s -> str.startsWith(s);
21:
22: System.out.println(methodRef.beginningCheck("A")); // false
```

English: Line 19 shows that we want to call `str.startsWith()` and pass a single parameter to be supplied at runtime. This would be a nice way of filtering the data in a list.

Türkçe: Satır 19, `str.startsWith()` çağırmak ve runtime'da sağlanacak tek bir parameter geçirmek istediğimizi gösterir. Bu, listedeki veriyi filter etmek için güzel bir yol olabilir.

English: A method reference doesn't have to take any parameters. In this example, we create a functional interface with a method that doesn't take any parameters but returns a value:

Türkçe: Method reference'ın parameter alması zorunlu değildir. Bu örnekte parameter almayan fakat değer döndüren bir method'u bulunan functional interface oluşturuyoruz:

```
interface StringChecker {
    boolean check();
}
```

English: We implement it by checking if the `String` is empty:

Türkçe: Bunu `String`'in empty olup olmadığını kontrol ederek implement ediyoruz:

```
18: var str = "";
19: StringChecker methodRef = str::isEmpty;
20: StringChecker lambda = () -> str.isEmpty();
21:
22: System.out.print(methodRef.check()); // true
```

English: Since the method on `String` is an instance method, we call the method reference on an instance of the `String` class.

Türkçe: `String` üzerindeki method bir instance method olduğundan method reference'ı `String` class'ının bir instance'ı üzerinde çağırırız.

English: While all method references can be turned into lambdas, the opposite is not always true. For example, consider this code:

Türkçe: Bütün method reference'lar lambda'ya dönüştürülebilse de bunun tersi her zaman doğru değildir. Örneğin şu kodu düşünün:

```
var str = "";
StringChecker lambda = () -> str.startsWith("Zoo");
```

English: How might we write this as a method reference? You might try one of the following:

Türkçe: Bunu method reference olarak nasıl yazabiliriz? Aşağıdakilerden birini deneyebilirsiniz:

```
StringChecker methodReference = str::startsWith; // DOES NOT COMPILE
StringChecker methodReference = str::startsWith("Zoo"); // DOES NOT COMPILE
```

English: Neither of these works! While we can pass the `str` as part of the method reference, there's no way to pass the `"Zoo"` parameter with it. Therefore, it is not possible to write this lambda as a method reference.

Türkçe: Bunların hiçbirisi çalışmaz! `str`'yi method reference'ın parçası olarak geçirebilsek de `"Zoo"` parameter'ını onunla birlikte geçirmenin bir yolu yoktur. Dolayısıyla bu lambda'yı method reference olarak yazmak mümkün değildir.

Calling Instance Methods on a Parameter

English: This time, we are going to call the same instance method that doesn't take any parameters. The trick is that we will do so without knowing the instance in advance. We need a different functional interface this time since it needs to know about the `String`:

Türkçe: Bu kez parameter almayan aynı instance method'u çağıracağız. Buradaki püf noktası, bunu instance'ı önceden bilmeden yapmamızdır. Bu kez `String` hakkında bilgi taşıması gerektiğinden farklı bir functional interface'e ihtiyacımız vardır:

```
interface StringParameterChecker {
    boolean check(String text);
}
```

English: We can implement this functional interface as follows:

Türkçe: Bu functional interface'i şöyle implement edebiliriz:

```
23: StringParameterChecker methodRef = String::isEmpty;  
24: StringParameterChecker lambda = s -> s.isEmpty();  
25:  
26: System.out.println(methodRef.check("Zoo")); // false
```

English: Line 23 says the method that we want to call is declared in `String`. It looks like a static method, but it isn't. Instead, Java knows that `isEmpty()` is an instance method that does not take any parameters. Java uses the parameter supplied at runtime as the instance on which the method is called.

Türkçe: Satır 23, çağırmak istediğimiz method'un `String` içinde bildirildiğini söyler. Static method gibi görünür; ancak değildir. Java, `isEmpty()` nin parameter almayan bir instance method olduğunu bilir. Runtime'da sağlanan parameter'ı method'un çağrılacağı instance olarak kullanır.

English: Compare lines 23 and 24 with lines 19 and 20 of our instance example. They look similar, although one references a local variable named `str`, while the other only references the functional interface parameters.

Türkçe: Satır 23 ve 24'ü instance örneğimizdeki satır 19 ve 20 ile karşılaştırın. Birisi `str` adlı local variable'a reference verirken diğeri yalnızca functional interface parameter'larına reference verse de benzer görünürler.

English: You can even combine the two types of instance method references. Again, we need a new functional interface that takes two parameters:

Türkçe: İki instance method reference türünü birleştirebilirsiniz. Yine iki parameter alan yeni bir functional interface'e ihtiyacımız vardır:

```
interface StringTwoParameterChecker {  
    boolean check(String text, String prefix);  
}
```

English: Pay attention to the parameter order when reading the implementation:

Türkçe: Implementation'ı okurken parameter sırasına dikkat edin:

```
26: StringTwoParameterChecker methodRef = String::startsWith;  
27: StringTwoParameterChecker lambda = (s, p) -> s.startsWith(p);  
28:  
29: System.out.println(methodRef.check("Zoo", "A")); // false
```

English: Since the functional interface takes two parameters, Java has to figure out what they represent. The first one will always be the instance of the object for instance methods. Any others are to be method parameters.

Türkçe: Functional interface iki parameter aldığından Java bunların neyi temsil ettiğini çözmelidir. Instance method'lar için ilki her zaman object'in instance'ı olur. Diğerleri method parameter'larıdır.

English: Remember that line 26 may look like a static method, but it is really a method reference declaring that the instance of the object will be specified later. Line 27 shows some of the power of a method reference. We were able to replace two lambda parameters this time.

Türkçe: Satır 26'nın static method gibi görünebileceğini; fakat gerçekte object instance'ının daha sonra belirtileceğini bildiren bir method reference olduğunu unutmayın. Satır 27 method reference'ın gücünün bir kısmını gösterir. Bu kez iki lambda parameter'ının yerini alabildik.

Calling Constructors

English: A constructor reference is a special type of method reference that uses `new` instead of a method and instantiates an object. For this example, our functional interface will not take any parameters but will return a `String`:

Türkçe: Constructor reference, bir method yerine `new` kullanan ve object instantiate eden özel bir method reference türüdür. Bu örnekte functional interface'imiz hiç parameter almayacak fakat `String` döndürecektir:

```
interface EmptyStringCreator {  
    String create();  
}
```

English: To call this, we use `new` as if it were a method name:

Türkçe: Bunu çağırmak için `new` ifadesini bir method adımı gibi kullanırız:

```
30: EmptyStringCreator methodRef = String::new;  
31: EmptyStringCreator lambda = () -> new String();  
32:  
33: var myString = methodRef.create();  
34: System.out.println(myString.equals("Snake")); // false
```

English: It expands like the method references you have seen so far. In the previous example, the lambda doesn't have any parameters.

Türkçe: Şimdiye kadar gördüğünüz method reference'lar gibi genişletilir. Önceki örnekte lambda'nın hiç parameter'ı yoktur.

English: Method references can be tricky. This time we create a functional interface that takes one parameter and returns a result:

Türkçe: Method reference'lar yanıltıcı olabilir. Bu kez bir parameter alan ve sonuç döndüren functional interface oluşturuyoruz:

```
interface StringCopier {  
    String copy(String value);  
}
```

English: In the implementation, notice that line 32 in the following example has the same method reference as line 30 in the previous example:

Türkçe: Implementation'da aşağıdaki örneğin 32. satırındaki method reference'ın önceki örneğin 30. satırındakiyle aynı olduğuna dikkat edin:

```

32: StringCopier methodRef = String::new;
33: StringCopier lambda = x -> new String(x);
34:
35: var myString = methodRef.copy("Zebra");
36: System.out.println(myString.equals("Zebra")); // true

```

English: This means you can't always determine which method can be called by looking at the method reference. Instead, you have to look at the context to see what parameters are used and if there is a return type. In this example, Java sees that we are passing a `String` parameter and calls the constructor of `String` that takes such a parameter.

Türkçe: Bu, yalnız method reference'a bakarak hangi method'un çağrılabilirliğini her zaman belirleyemeyeceğiniz anlamına gelir. Bunun yerine hangi parameter'ların kullanıldığını ve return type bulunup bulunmadığını görmek için context'e bakmalısınız. Bu örnekte Java bir `String` parameter geçirdiğimizi görür ve `String`'in böyle bir parameter alan constructor'ını çağırır.

Reviewing Method References

English: Reading method references is helpful in understanding the code. Table 8.3 shows the four types of method references. If this table doesn't make sense, please reread the previous section. It can take a few tries before method references start to add up.

Türkçe: Method reference'ları okumak kodu anlamaya yardımcı olur. Table 8.3 dört method reference türünü gösterir. Bu tablo anlamlı gelmiyorsa önceki bölümü yeniden okuyun. Method reference'ların anlam kazanmaya başlaması birkaç deneme gerektirebilir.

Table 8.3 · Method references

Type / Tür	Before colon / :: öncesi	After colon / :: sonrası	Example / Örnek
static methods / static method'lar	Class name / Class adı	Method name / Method adı	<code>Math::random</code>
Instance methods on a particular object / Belirli bir object üzerindeki instance method'lar	Instance variable name / Instance variable adı	Method name / Method adı	<code>str::startsWith</code>
Instance methods on a parameter / Bir parameter üzerindeki instance method'lar	Class name / Class adı	Method name / Method adı	<code>String::isEmpty</code>
Constructor / Kurucu	Class name / Class adı	<code>new</code>	<code>String::new</code>

Working with Built-in Functional Interfaces

English: It would be inconvenient to write your own functional interface any time you want to write a lambda. Luckily, a large number of general-purpose functional interfaces are provided for you. We cover them in this section.

Türkçe: Her lambda yazmak istediğinizde kendi functional interface'inizi yazmak zahmetli olurdu. Neyse ki çok sayıda general-purpose functional interface hazır olarak sunulur. Bu bölümde bunları ele alıyoruz.

English: The core functional interfaces in Table 8.4 are provided in the `java.util.function` package. We cover generics in the next chapter, but for now, you just need to know that `<T>` allows the interface to take an object of a specified type. If a second type parameter is needed, we use the next letter, `U`. If a distinct return type is needed, we choose `R` for return as the generic type.

Türkçe: Table 8.4'teki core functional interface'ler `java.util.function` package'ında sağlanır. Generics konusunu sonraki bölümde ele alacağız; şimdilik yalnızca `<T>`'nin interface'in belirtilen bir type'ta object almasına olanak tanıdığını bilmelisiniz. İkinci bir type parameter gerekiyorsa sonraki harf `U`yu kullanırız. Farklı bir return type gerekiyorsa return sözcüğünü temsilen generic type olarak `R`yi seçeriz.

English: Table 8.4 lists common functional interfaces.

Türkçe: Table 8.4 yaygın functional interface'leri listeler.

Table 8.4 · Common functional interfaces · Part 1

Functional interface / Fonksiyonel arayüz	Return type / Dönüş tipi	Method name / Method adı	Number of parameters / Parameter sayısı
<code>Supplier<T></code>	<code>T</code>	<code>get()</code>	0
<code>Consumer<T></code>	<code>void</code>	<code>accept(T)</code>	1 (T)
<code>BiConsumer<T, U></code>	<code>void</code>	<code>accept(T, U)</code>	2 (T, U)
<code>Predicate<T></code>	<code>boolean</code>	<code>test(T)</code>	1 (T)
<code>BiPredicate<T, U></code>	<code>boolean</code>	<code>test(T, U)</code>	2 (T, U)
<code>Function<T, R></code>	<code>R</code>	<code>apply(T)</code>	1 (T)

Table 8.4 · Common functional interfaces · Part 2

Functional interface / Fonksiyonel arayüz	Return type / Dönüş tipi	Method name / Method adı	Number of parameters / Parameter sayısı
<code>BiFunction<T, U, R></code>	<code>R</code>	<code>apply(T, U)</code>	2 (T, U)
<code>UnaryOperator<T></code>	<code>T</code>	<code>apply(T)</code>	1 (T)
<code>BinaryOperator<T></code>	<code>T</code>	<code>apply(T, T)</code>	2 (T, T)

English: For the exam, you need to memorize Table 8.4. We will give you lots of practice in this section to help make it memorable. Before you ask, most of the time we don't assign the implementation of the interface to a variable. The interface name is implied, and it is passed directly to the method that needs it. We are introducing the names so that you can better understand and remember what is going on. By the next chapter, we will assume that you have this down and stop creating the intermediate variable.

Türkçe: Sınav için Table 8.4'ü ezberlemeniz gerekir. Akılda kalmasına yardımcı olmak için bu bölümde bolca pratik sunacağız. Siz sormadan belirtelim: Çoğu zaman interface implementation'ını bir variable'a atamayız. Interface adı context'ten anlaşılır ve doğrudan ona ihtiyaç duyan method'a geçirilir. Neler olduğunu daha iyi anlayıp hatırlayabilmeniz için adları tanıtıyoruz. Sonraki bölüme geldiğimizde bunu öğrendiğinizi varsayıp intermediate variable oluşturmayı bırakacağız.

English: You learn about a few more functional interfaces later in the book. In the next chapter, we cover `Comparator`. In Chapter 13, “Concurrency,” we discuss `Runnable` and `Callable`. These may show up on the exam when you are asked to recognize functional interfaces.

Türkçe: Kitabın ilerleyen bölümlerinde birkaç functional interface daha öğreneceksiniz. Sonraki bölümde `Comparator`’ı ele alıyoruz. Chapter 13, “Concurrency” bölümünde `Runnable` ve `Callable`’i tartışıyoruz. Functional interface’leri tanımanız istendiğinde bunlar sınavda karşınıza çıkabilir.

English: Let’s look at how to implement each of these interfaces. Since both lambdas and method references appear all over the exam, we show an implementation using both where possible. After introducing the interfaces, we also cover some convenience methods available on these interfaces.

Türkçe: Bu interface’lerin her birinin nasıl implement edildiğine bakalım. Hem lambda’lar hem method reference’lar sınavın her yerinde karşınıza çıktığından mümkün olduğunda ikisini de kullanan implementation gösteriyoruz. Interface’leri tanıttıktan sonra bunlardaki bazı convenience method’ları da ele alıyoruz.

Implementing Supplier

English: A `Supplier` is used when you want to generate or supply values without taking any input. The `Supplier` interface is defined as follows:

Türkçe: Herhangi bir input almadan değer üretmek veya sağlamak istediğinizde `Supplier` kullanılır. `Supplier` interface’i şöyle tanımlanır:

```
@FunctionalInterface
public interface Supplier<T> {
    T get();
}
```

English: You can create a `LocalDate` object using the factory method `now()`. This example shows how to use a `Supplier` to call this factory:

Türkçe: `now()` factory method’unu kullanarak `LocalDate` object’i oluşturabilirsiniz. Bu örnek, bu factory’yi çağırarak için `Supplier`’ın nasıl kullanılacağını gösterir:

```
Supplier<LocalDate> s1 = LocalDate::now;
Supplier<LocalDate> s2 = () -> LocalDate.now();
LocalDate d1 = s1.get();
LocalDate d2 = s2.get();
```

```
System.out.println(d1); // 2022-02-20
System.out.println(d2); // 2022-02-20
```

OCP notu: `LocalDate.now()` sistem tarihini kullanır. Kaynaktaki `2022-02-20` sabit bir Java sonucu değil, örnek çalıştırmanın output’udur.

English: This example prints a date twice. It's also a good opportunity to review static method references. The `LocalDate::now` method reference is used to create a `Supplier` to assign to an intermediate variable `s1`. A `Supplier` is often used when constructing new objects. For example, we can print two empty `StringBuilder` objects:

Türkçe: Bu örnek bir tarihi iki kez yazdırır. Aynı zamanda static method reference'ları gözden geçirmek için iyi bir fırsattır. `LocalDate::now` method reference'ı, intermediate variable `s1` e atanacak bir `Supplier` oluşturmak için kullanılır. `Supplier` çoğunlukla yeni object'ler oluştururken kullanılır. Örneğin iki empty `StringBuilder` object'i yazdırabiliriz:

```
Supplier<StringBuilder> s1 = StringBuilder::new;
Supplier<StringBuilder> s2 = () -> new StringBuilder();
System.out.println(s1.get()); // Empty string
System.out.println(s2.get()); // Empty string
```

English: This time, we used a constructor reference to create the object. We've been using generics to declare what type of `Supplier` we are using. This can be a little long to read. Can you figure out what the following does? Just take it one step at a time:

Türkçe: Bu kez object'i oluşturmak için constructor reference kullandık. Hangi `Supplier` type'ını kullandığımızı bildirmek için generics kullanıyoruz. Bunu okumak biraz uzun olabilir. Aşağıdakinin ne yaptığını çözebilir misiniz? Adım adım ilerleyin:

```
Supplier<ArrayList<String>> s3 = ArrayList::new;
ArrayList<String> a1 = s3.get();
System.out.println(a1); // []
```

English: We have a `Supplier` of a certain type. That type happens to be `ArrayList<String>`. Then calling `get()` creates a new instance of `ArrayList<String>`, which is the generic type of the `Supplier`—in other words, a generic that contains another generic. Be sure to look at the code carefully when this type of thing comes up.

Türkçe: Belirli bir type'ta `Supplier`ımız vardır. Bu type `ArrayList<String>` dir. Ardından `get()` çağırısı `Supplier`ın generic type'ı olan yeni bir `ArrayList<String>` instance'ı oluşturur; başka bir deyişle bu, başka bir generic içeren generic'tir. Bu tür bir durumla karşılaştığınızda koda dikkatle baktığınızdan emin olun.

English: Notice how we called `get()` on the functional interface. What would happen if we tried to print out `s3` itself?

Türkçe: Functional interface üzerinde `get()` i nasıl çağırdığımıza dikkat edin. `s3` ün kendisini yazdırmaya çalışsaydık ne olurdu?

```
System.out.println(s3);
```

English: The code prints something like this:

Türkçe: Kod aşağıdakine benzer bir şey yazdırır:

```
functionalinterface.BuiltIns$$Lambda$1/0x0000000800066840@4909b8da
```

English: That's the result of calling `toString()` on a lambda. Yuck. This actually does mean something. Our test class is named `BuiltIns`, and it is in a package that we created named `functionalinterface`. Then comes `$$`, which means that the class doesn't exist in a class file on the file system. It exists only in memory. You don't need to worry about the rest.

Türkçe: Bu, lambda üzerinde `toString()` çağırmanın sonucudur. Pek hoş değil. Aslında bir anlamı vardır. Test class'ımızın adı `BuiltIns`tır ve oluşturduğumuz `functionalinterface` adlı package'ın içindedir. Ardından gelen `$$`, class'ın file system üzerinde bir class file içinde bulunmadığı; yalnız memory'de var olduğu anlamına gelir. Geri kalanı için endişelenmenize gerek yoktur.

Java 17 notu: Lambda object'inin generated class adı ve `toString()` görünümü implementation detail'dir; portable bir output olarak varsayılmamalıdır.

Implementing Consumer and BiConsumer

English: You use a `Consumer` when you want to do something with a parameter but not return anything. `BiConsumer` does the same thing, except that it takes two parameters. The interfaces are defined as follows:

Türkçe: Bir parameter ile bir şey yapmak ancak hiçbir şey döndürmemek istediğinizde `Consumer` kullanırsınız. `BiConsumer` aynı şeyi yapar; tek farkı iki parameter almasıdır. Interface'ler şöyle tanımlanır:

```
@FunctionalInterface
public interface Consumer<T> {
```

```
    void accept(T t);
    // omitted default method
}
```

```
@FunctionalInterface
public interface BiConsumer<T, U> {
    void accept(T t, U u);
    // omitted default method
}
```

English: You'll notice this pattern. `Bi` means two. It comes from Latin, but you can remember it from English words like binary (0 or 1) or bicycle (two wheels). Always add another parameter when you see `Bi`.

Türkçe: Bu pattern'ı fark edeceksiniz. `Bi` iki demektir. Latince kökenlidir; ancak binary (0 veya 1) ya da bicycle (iki tekerlek) gibi İngilizce sözcüklerden hatırlayabilirsiniz. `Bi` gördüğünüzde her zaman bir parameter daha ekleyin.

English: Printing is a common use of the `Consumer` interface:

Türkçe: Yazdırma, `Consumer` interface'inin yaygın bir kullanımıdır:

```
Consumer<String> c1 = System.out::println;
Consumer<String> c2 = x -> System.out.println(x);
c1.accept("Annie"); // Annie
c2.accept("Annie"); // Annie
```

English: `BiConsumer` is called with two parameters. They don't have to be the same type. For example, we can put a key and a value in a map using this interface:

Türkçe: `BiConsumer` iki parameter ile çağrılır. Bunların aynı type'ta olması gerekmez. Örneğin bu interface'i kullanarak bir map'e key ve value koyabiliriz:

```
var map = new HashMap<String, Integer>();
BiConsumer<String, Integer> b1 = map::put;
BiConsumer<String, Integer> b2 = (k, v) -> map.put(k, v);
b1.accept("chicken", 7);
b2.accept("chick", 1);
System.out.println(map); // {chicken=7, chick=1}
```

Java 17 notu: `HashMap` iteration order garanti etmez. Kaynaktaki `{chicken=7, chick=1}` gösterimi içerik açısından doğrudur; entry sırası portable output değildir.

English: The output is `{chicken=7, chick=1}`, which shows that both `BiConsumer` implementations were called. When declaring `b1`, we used an instance method reference on an object since we want to call a method on the local variable `map`. The code to instantiate `b1` is a good bit shorter than the code for `b2`. This is probably why the exam is so fond of method references.

Türkçe: Output `{chicken=7, chick=1}` olur; bu da iki `BiConsumer` implementation'ının da çağrıldığını gösterir. `b1`i bildirirken local variable `map` üzerinde method çağırmak istediğimiz için bir object üzerindeki instance method reference'ı kullandık. `b1`i instantiate eden kod `b2` kodundan oldukça kısadır. Sınavın method reference'ları çok sevmesinin nedeni muhtemelen budur.

English: As another example, we use the same type for both generic parameters:

Türkçe: Başka bir örnek olarak iki generic parameter için de aynı type'ı kullanıyoruz:

```
var map = new HashMap<String, String>();
BiConsumer<String, String> b1 = map::put;
BiConsumer<String, String> b2 = (k, v) -> map.put(k, v);
```

```
b1.accept("chicken", "Cluck");
b2.accept("chick", "Tweep");
System.out.println(map); // {chicken=Cluck, chick=Tweep}
```

Java 17 notu: Bu `HashMap` output'unda da entry sırası garanti edilmez.

English: This shows that a `BiConsumer` can use the same type for both the `T` and `U` generic parameters.

Türkçe: Bu, bir `BiConsumer`'ın hem `T` hem `U` generic parameter'ı için aynı type'ı kullanabileceğini gösterir.

Implementing Predicate and BiPredicate

English: `Predicate` is often used when filtering or matching. Both are common operations. A `BiPredicate` is just like a `Predicate`, except that it takes two parameters instead of one. The interfaces are defined as follows:

Türkçe: `Predicate` çoğunlukla filtering veya matching yaparken kullanılır. İkisi de yaygın operation'lardır. `BiPredicate`, tek parameter yerine iki parameter alması dışında `Predicate` gibidir. Interface'ler şöyle tanımlanır:

```
@FunctionalInterface
public interface Predicate<T> {
    boolean test(T t);
    // omitted default and static methods
}

@FunctionalInterface
public interface BiPredicate<T, U> {
    boolean test(T t, U u);
    // omitted default methods
}
```

English: You can use a `Predicate` to test a condition.

Türkçe: Bir condition'ı test etmek için `Predicate` kullanabilirsiniz.

```
Predicate<String> p1 = String::isEmpty;
Predicate<String> p2 = x -> x.isEmpty();
System.out.println(p1.test("")); // true
System.out.println(p2.test("")); // true
```

English: This prints `true` twice. More interesting is a `BiPredicate`. This example also prints `true` twice:

Türkçe: Bu kod iki kez `true` yazdırır. `BiPredicate` daha ilginçtir. Şu örnek de iki kez `true` yazdırır:

```
BiPredicate<String, String> b1 = String::startsWith;
BiPredicate<String, String> b2 =
    (string, prefix) -> string.startsWith(prefix);
System.out.println(b1.test("chicken", "chick")); // true
System.out.println(b2.test("chicken", "chick")); // true
```

English: The method reference includes both the instance variable and parameter for `startsWith()`. This is a good example of how method references save quite a lot of typing. The downside is that they are less explicit, and you really have to understand what is going on!

Türkçe: Method reference, `startsWith()` için hem instance variable'ı hem parameter'ı içerir. Bu, method reference'ların yazma işini ne kadar azalttığına iyi bir örnektir. Dezavantajı, daha az explicit olmaları ve gerçekten neler olduğunu anlamayı gerektirmeleridir!

Implementing Function and BiFunction

English: A **Function** is responsible for turning one parameter into a value of a potentially different type and returning it. Similarly, a **BiFunction** is responsible for turning two parameters into a value and returning it. The interfaces are defined as follows:

Türkçe: **Function**, bir parameter'ı potansiyel olarak farklı bir type'taki değere dönüştürüp döndürmekten sorumludur. Benzer şekilde **BiFunction**, iki parameter'ı bir değere dönüştürüp döndürmekten sorumludur. Interface'ler şöyle tanımlanır:

```
@FunctionalInterface
public interface Function<T, R> {
    R apply(T t);
    // omitted default and static methods
}

@FunctionalInterface
public interface BiFunction<T, U, R> {
    R apply(T t, U u);
    // omitted default method
}
```

English: For example, this function converts a **String** to the length of the **String**:

Türkçe: Örneğin bu function bir **String**i o **String**in length değerine dönüştürür:

```
Function<String, Integer> f1 = String::length;
Function<String, Integer> f2 = x -> x.length();
System.out.println(f1.apply("cluck")); // 5
System.out.println(f2.apply("cluck")); // 5
```

English: This function turns a **String** into an **Integer**. Well, technically, it turns the **String** into an **int**, which is autoboxed into an **Integer**. The types don't have to be different. The following combines two **String** objects and produces another **String**:

Türkçe: Bu function bir **String**i **Integer**a dönüştürür. Teknik olarak **String**i **int**e dönüştürür ve bu değer bir **Integer**a autobox edilir. Type'ların farklı olması gerekmez. Aşağıdaki kod iki **String** object'ini birleştirip başka bir **String** üretir:

```
BiFunction<String, String, String> b1 = String::concat;
BiFunction<String, String, String> b2 =
    (string, toAdd) -> string.concat(toAdd);
System.out.println(b1.apply("baby ", "chick")); // baby chick
System.out.println(b2.apply("baby ", "chick")); // baby chick
```

English: The first two types in the **BiFunction** are the input types. The third is the result type. For the method reference, the first parameter is the instance that **concat()** is called on, and the second is passed to **concat()**.

Türkçe: **BiFunction**daki ilk iki type input type'larıdır. Üçüncü type result type'tır. Method reference için ilk parameter **concat()** in çağrıldığı instance; ikinci parameter ise **concat()** a geçirilen değerdir.

Implementing UnaryOperator and BinaryOperator

English: `UnaryOperator` and `BinaryOperator` are special cases of a `Function`. They require all type parameters to be the same type. A `UnaryOperator` transforms its value into one of the same type. For example, incrementing by one is a unary operation. In fact, `UnaryOperator` extends `Function`. A `BinaryOperator` merges two values into one of the same type. Adding two numbers is a binary operation. Similarly, `BinaryOperator` extends `BiFunction`. The interfaces are defined as follows:

Türkçe: `UnaryOperator` ve `BinaryOperator`, `Function`’ın özel durumlarıdır. Bütün type parameter’ların aynı type olmasını gerektirirler. `UnaryOperator` değerini aynı type’ta başka bir değere dönüştürür. Örneğin bir artırmak unary operation’dır. Aslında `UnaryOperator`, `Function`’ı extend eder. `BinaryOperator` iki değeri aynı type’ta tek bir değer hâlinde birleştirir. İki sayıyı toplamak binary operation’dır. Benzer biçimde `BinaryOperator`, `BiFunction`’ı extend eder. Interface’ler şöyle tanımlanır:

```
@FunctionalInterface
public interface UnaryOperator<T> extends Function<T, T> {
    // omitted static method
}

@FunctionalInterface
public interface BinaryOperator<T> extends BiFunction<T, T, T> {
    // omitted static methods
}
```

English: This means the method signatures look like this:

Türkçe: Bu, method signature’ların şöyle görüldüğü anlamına gelir:

```
T apply(T t);           // UnaryOperator
T apply(T t1, T t2);    // BinaryOperator
```

English: In the Javadoc, you’ll notice that these methods are inherited from the `Function`/`BiFunction` superclass. The generic declarations on the subclass are what force the type to be the same. For the unary example, notice how the return type is the same type as the parameter.

Türkçe: Javadoc’ta bu method’ların `Function`/`BiFunction` superclass’ından inherit edildiğini göreceksiniz. Subclass üzerindeki generic declaration’lar type’ı aynı olmaya zorlar. Unary örnekte return type’ın parameter ile aynı type olduğuna dikkat edin.

Editor note: Source text says “superclass” and “subclass.” Technically, `Function` and `BiFunction` are superinterfaces, while `UnaryOperator` and `BinaryOperator` are subinterfaces.

Editör notu: Kaynak “superclass” ve “subclass” der. Teknik olarak `Function` ile `BiFunction` superinterface; `UnaryOperator` ile `BinaryOperator` ise subinterface’tir.

```
UnaryOperator<String> u1 = String::toUpperCase;
UnaryOperator<String> u2 = x -> x.toUpperCase();
System.out.println(u1.apply("chirp")); // CHIRP
System.out.println(u2.apply("chirp")); // CHIRP
```

English: This prints **CHIRP** twice. We don't need to specify the return type in the generics because **UnaryOperator** requires it to be the same as the parameter. And now here's the binary example:

Türkçe: Bu kod iki kez **CHIRP** yazdırır. **UnaryOperator** return type'ın parameter ile aynı olmasını gerektirdiğinden generics içinde return type'ı belirtmemize gerek yoktur. Şimdi de binary örneğe bakalım:

```
BinaryOperator<String> b1 = String::concat;
BinaryOperator<String> b2 = (string, toAdd) -> string.concat(toAdd);
System.out.println(b1.apply("baby ", "chick")); // baby chick
System.out.println(b2.apply("baby ", "chick")); // baby chick
```

English: Notice that this does the same thing as the **BiFunction** example. The code is more succinct, which shows the importance of using the best functional interface. It's nice to have one generic type specified instead of three.

Türkçe: Bunun **BiFunction** örneğiyle aynı şeyi yaptığını dikkat edin. Kod daha succinct'tir; bu da en uygun functional interface'i kullanmanın önemini gösterir. Üç generic type yerine bir tane belirtmek güzeldir.

Checking Functional Interfaces

English: It's really important to know the number of parameters, types, return value, and method name for each of the functional interfaces. Now would be a good time to memorize Table 8.4 if you haven't done so already. Let's do some examples to practice.

Türkçe: Functional interface'lerin her biri için parameter sayısını, type'ları, return value'yu ve method adını bilmek gerçekten önemlidir. Henüz yapmadıysanız Table 8.4'ü ezberlemek için tam zamanıdır. Pratik yapmak üzere birkaç örneğe bakalım.

English: What functional interface would you use in these three situations? Returns a **String** without taking any parameters. Returns a **Boolean** and takes a **String**. Returns an **Integer** and takes two **Integers**.

Türkçe: Şu üç durumda hangi functional interface'i kullanırdınız? Hiç parameter almadan **String** döndürür. Bir **String** alıp **Boolean** döndürür. İki **Integer** alıp **Integer** döndürür.

English: Ready? Think about what your answers are before continuing. Really. You have to know this cold. Okay. The first one is a **Supplier<String>** because it generates an object and takes zero parameters. The second one is a **Function<String, Boolean>** because it takes one parameter and returns another type. It's a little tricky. You might think it is a **Predicate<String>**. Note that a **Predicate** returns a **boolean** primitive and not a **Boolean** object.

Türkçe: Hazır mısınız? Devam etmeden önce cevaplarınızı düşünün. Gerçekten bunu çok iyi bilmelisiniz. İlki object üretip sıfır parameter aldığı için **Supplier<String>**dir. İkincisi bir parameter alıp başka bir type döndürdüğü için **Function<String, Boolean>**dir. Biraz yanıltıcıdır; **Predicate<String>** olduğunu düşünebilirsiniz. **Predicate**in **Boolean** object değil **boolean** primitive döndürdüğüne dikkat edin.

English: Finally, the third one is either a `BinaryOperator<Integer>` or a `BiFunction<Integer, Integer, Integer>`. Since `BinaryOperator` is a special case of `BiFunction`, either is a correct answer. `BinaryOperator<Integer>` is the better answer of the two since it is more specific.

Türkçe: Sonuncusu ya `BinaryOperator<Integer>` ya da `BiFunction<Integer, Integer, Integer>` dir. `BinaryOperator`, `BiFunction` ın özel bir durumu olduğundan ikisi de doğru cevaptır. `BinaryOperator<Integer>` daha specific olduğu için ikisinden daha iyi cevaptır.

English: Let's try this exercise again but with code. It's harder with code. The first thing you do is look at how many parameters the lambda takes and whether there is a return value. What functional interface would you use to fill in the blanks for these?

Türkçe: Bu alıştırmayı kodla yeniden deneyelim. Kodla daha zordur. İlk yapacağınız şey lambda'nın kaç parameter aldığına ve return value olup olmadığına bakmaktır. Aşağıdaki boşlukları doldurmak için hangi functional interface'i kullanırdınız?

```
6: _____ <List> ex1 = x -> "".equals(x.get(0));
7: _____ <Long> ex2 = (Long l) -> System.out.println(l);
8: _____ <String, String> ex3 = (s1, s2) -> false;
```

English: Again, think about the answers before continuing. Ready? Line 6 passes one `List` parameter to the lambda and returns a `boolean`. This tells us that it is a `Predicate` or `Function`. Since the generic declaration has only one parameter, it is a `Predicate`.

Türkçe: Yine devam etmeden önce cevapları düşünün. Hazır mısınız? Satır 6 lambda'ya bir `List` parameter geçirir ve `boolean` döndürür. Bu bize onun `Predicate` veya `Function` olduğunu söyler. Generic declaration yalnızca bir parameter içerdiğinden `Predicate` tir.

English: Line 7 passes one `Long` parameter to the lambda and doesn't return anything. This tells us that it is a `Consumer`. Line 8 takes two parameters and returns a `boolean`. When you see a `boolean` returned, think `Predicate` unless the generics specify a `Boolean` return type. In this case, there are two parameters, so it is a `BiPredicate`.

Türkçe: Satır 7 lambda'ya bir `Long` parameter geçirir ve hiçbir şey döndürmez. Bu bize onun `Consumer` olduğunu söyler. Satır 8 iki parameter alıp `boolean` döndürür. Generics bir `Boolean` return type belirtmediği sürece dönen bir `boolean` gördüğünüzde `Predicate` düşünün. Burada iki parameter bulunduğu için `BiPredicate` tir.

English: Are you finding these easy? If not, review Table 8.4 again. We aren't kidding. You need to know the table really well. Now that you are fresh from studying the table, we are going to play "identify the error." These are meant to be tricky:

Türkçe: Bunları kolay buluyor musunuz? Bulmuyorsanız Table 8.4'ü tekrar gözden geçirin. Şaka yapmıyoruz; tabloyu gerçekten çok iyi bilmelisiniz. Tabloyu yeni çalışmışken şimdi "hatayı belirle" oyunu oynayacağız. Bunlar yanıltıcı olacak şekilde hazırlanmıştır:

```
6: Function<List<String>> ex1 = x -> x.get(0); // DOES NOT COMPILE
7: UnaryOperator<Long> ex2 = (Long l) -> 3.14; // DOES NOT COMPILE
```

English: Line 6 claims to be a `Function`. A `Function` needs to specify two generic types: the input parameter type and the return value type. The return value type is missing from line 6, causing the code not to compile. Line 7 is a `UnaryOperator`, which returns the same type as it is passed in. The example returns a `double` rather than a `Long`, causing the code not to compile.

Türkçe: Satır 6 bir `Function` olduğunu iddia eder. `Function` iki generic type belirtmelidir: input parameter type ve return value type. Satır 6'da return value type eksik olduğundan kod derlenmez. Satır 7, kendisine geçirilen type ile aynı type'ı döndüren bir `UnaryOperator`dir. Örnek `Long` yerine `double` döndürdüğü için kod derlenmez.

Using Convenience Methods on Functional Interfaces

English: By definition, all functional interfaces have a single abstract method. This doesn't mean they can have only one method, though. Several of the common functional interfaces provide a number of helpful default interface methods.

Türkçe: Tanım gereği bütün functional interface'lerin tek bir abstract method'u vardır. Ancak bu, yalnızca bir method'a sahip olabilecekleri anlamına gelmez. Yaygın functional interface'lerin birçoğu çeşitli yararlı default interface method'ları sunar.

English: Table 8.5 shows the convenience methods on the built-in functional interfaces that you need to know for the exam. All of these facilitate modifying or combining functional interfaces of the same type. Note that Table 8.5 shows only the main interfaces. The `BiConsumer`, `BiFunction`, and `BiPredicate` interfaces have similar methods available.

Türkçe: Table 8.5, sınav için bilmeniz gereken built-in functional interface convenience method'larını gösterir. Bunların tümü aynı type'taki functional interface'leri değiştirmeyi veya birleştirmeyi kolaylaştırır. Table 8.5'in yalnızca ana interface'leri gösterdiğine dikkat edin. `BiConsumer`, `BiFunction` ve `BiPredicate` interface'lerinde benzer method'lar bulunur.

Table 8.5 · Convenience methods

Interface instance / Interface instance'i	Method return type / Method dönüş tipi	Method name / Method adı	Method parameters / Method parameter'ları
<code>Consumer</code>	<code>Consumer</code>	<code>andThen()</code>	<code>Consumer</code>
<code>Function</code>	<code>Function</code>	<code>andThen()</code>	<code>Function</code>
<code>Function</code>	<code>Function</code>	<code>compose()</code>	<code>Function</code>
<code>Predicate</code>	<code>Predicate</code>	<code>and()</code>	<code>Predicate</code>
<code>Predicate</code>	<code>Predicate</code>	<code>negate()</code>	—
<code>Predicate</code>	<code>Predicate</code>	<code>or()</code>	<code>Predicate</code>

English: Let's start with these two `Predicate` variables:

Türkçe: Şu iki `Predicate` variable ile başlayalım:

```
Predicate<String> egg = s -> s.contains("egg");
Predicate<String> brown = s -> s.contains("brown");
```

English: Now we want a `Predicate` for brown eggs and another for all other colors of eggs. We could write this by hand, as shown here:

Türkçe: Şimdi kahverengi yumurtalar için bir `Predicate`, diğer bütün renkteki yumurtalar için de başka bir `Predicate` istiyoruz. Bunu burada gösterildiği gibi elle yazabiliriz:

```
Predicate<String> brownEggs =
    s -> s.contains("egg") && s.contains("brown");
Predicate<String> otherEggs =
    s -> s.contains("egg") && !s.contains("brown");
```

English: This works, but it's not great. It's a bit long to read, and it contains duplication. What if we decide the letter `e` should be capitalized in `egg`? We'd have to change it in three variables: `egg`, `brownEggs`, and `otherEggs`. A better way to deal with this situation is to use two of the default methods on `Predicate`.

Türkçe: Bu çalışır; fakat pek iyi değildir. Okuması biraz uzundur ve duplication içerir. `egg` içindeki `e` harfinin büyük olması gerektiğine karar verirsek ne olur? Bunu üç variable'da değiştirmemiz gerekir: `egg`, `brownEggs` ve `otherEggs`. Bu durumla baş etmenin daha iyi yolu, `Predicate` üzerindeki default method'lardan ikisini kullanmaktır.

```
Predicate<String> brownEggs = egg.and(brown);
Predicate<String> otherEggs = egg.and(brown.negate());
```

English: Neat! Now we are reusing the logic in the original `Predicate` variables to build two new ones. It's shorter and clearer what the relationship is between variables. We can also change the spelling of `egg` in one place, and the other two objects will have new logic because they reference it.

Türkçe: Harika! Artık iki yeni `Predicate` oluşturmak için özgün `Predicate` variable'larındaki logic'i yeniden kullanıyoruz. Daha kısadır ve variable'lar arasındaki ilişki daha açıktır. Ayrıca `egg` yazımını tek bir yerde değiştirebiliriz; diğer iki object ona reference verdiği için yeni logic'e sahip olur.

English: Moving on to `Consumer`, let's take a look at the `andThen()` method, which runs two functional interfaces in sequence:

Türkçe: `Consumer`a geçerek iki functional interface'i sırayla çalıştıran `andThen()` method'una bakalım:

```
Consumer<String> c1 = x -> System.out.print("1: " + x);
Consumer<String> c2 = x -> System.out.print(",2: " + x);
Consumer<String> combined = c1.andThen(c2);
combined.accept("Annie"); // 1: Annie,2: Annie
```

English: Notice how the same parameter is passed to both `c1` and `c2`. This shows that the `Consumer` instances are run in sequence and are independent of each other. By contrast, the `compose()` method on `Function` chains functional interfaces. However, it passes along the output of one to the input of another.

Türkçe: Aynı parameter'ın hem `c1`e hem `c2`ye geçirildiğine dikkat edin. Bu, `Consumer` instance'larının sırayla çalıştırıldığını ve birbirlerinden bağımsız olduğunu gösterir. Buna karşılık `Function` üzerindeki `compose()` method'u functional interface'leri zincirler; birinin output'unu diğerinin input'una aktarır.

```
Function<Integer, Integer> before = x -> x + 1;
Function<Integer, Integer> after = x -> x * 2;
Function<Integer, Integer> combined = after.compose(before);
System.out.println(combined.apply(3)); // 8
```

English: This time, the `before` runs first, turning the `3` into `4`. Then the `after` runs, doubling the `4` to `8`. All of the methods in this section are helpful for simplifying your code as you work with functional interfaces.

Türkçe: Bu kez önce `before` çalışarak `3`ü `4`e dönüştürür. Ardından `after` çalışıp `4`ü ikiyle çarparak `8` yapar. Bu bölümdeki bütün method'lar functional interface'lerle çalışırken kodunuzu sadeleştirmeye yardımcı olur.

Learning the Functional Interfaces for Primitives

English: Remember when we told you to memorize Table 8.4 with the common functional interfaces? Did you? If you didn't, go do it now. We'll wait. We are about to make it more involved. There are also a large number of special functional interfaces for primitives. These are useful in Chapter 10 when we cover streams and optionals.

Türkçe: Yaygın functional interface'leri içeren Table 8.4'ü ezberlemenizi söylediğimizi hatırlıyor musunuz? Ezberlediniz mi? Ezberlemediyseniz şimdi yapın; bekleriz. Konuyu biraz daha ayrıntılı hâle getirmek üzereyiz. Primitive'ler için de çok sayıda özel functional interface vardır. Bunlar Chapter 10'da stream ve optional konularını ele alırken yararlı olacaktır.

English: Most of them are for the `double`, `int`, and `long` types. There is one exception, which is `BooleanSupplier`. We cover that before introducing the functional interfaces for `double`, `int`, and `long`.

Türkçe: Bunların çoğu `double`, `int` ve `long` type'ları içindir. Tek istisna `BooleanSupplier`dir. `double`, `int` ve `long` functional interface'lerini tanıtmadan önce onu ele alıyoruz.

Functional Interfaces for `boolean`

English: `BooleanSupplier` is a separate type. It has one method to implement:

Türkçe: `BooleanSupplier` ayrı bir type'tır. Implement edilecek tek bir method'u vardır:

```
@FunctionalInterface
public interface BooleanSupplier {
    boolean getAsBoolean();
}
```

English: It works just as you've come to expect from functional interfaces. Here's an example:

Türkçe: Functional interface'lerden beklemeye alıştığınız biçimde çalışır. İşte bir örnek:

```
12: BooleanSupplier b1 = () -> true;
13: BooleanSupplier b2 = () -> Math.random() > .5;
14: System.out.println(b1.getAsBoolean()); // true
15: System.out.println(b2.getAsBoolean()); // false
```

English: Lines 12 and 13 each create a `BooleanSupplier`, which is the only functional interface for `boolean`. Line 14 prints `true`, since it is the result of `b1`. Line 15 prints `true` or `false`, depending on the random value generated.

Türkçe: Satır 12 ve 13'ün her biri `boolean` için tek functional interface olan bir `BooleanSupplier` oluşturur. Satır 14, `b1`'in sonucu olduğu için `true` yazdırır. Satır 15 üretilen random değere bağlı olarak `true` veya `false` yazdırır.

Java 17 kapsam notu: Kaynaktaki “the only functional interface for `boolean`” ifadesi, `BooleanSupplier`’ın `Boolean...` adıyla sunulan özel zero-input primitive specialization olması bağlamında okunmalıdır. `Predicate<T>`, `BiPredicate<T, U>` ve `IntPredicate` gibi interface’lerin SAM method’ları da primitive `boolean` döndürür.

OCP notu: Satır 15'teki `// false` yalnızca örnek bir çalıştırmanın sonucudur; `Math.random()` nedeniyle output deterministic değildir.

Functional Interfaces for `double`, `int`, and `long`

English: Most of the functional interfaces are for `double`, `int`, and `long`. Table 8.6 shows the equivalent of Table 8.4 for these primitives. You probably won’t be surprised that you have to memorize it. Luckily, you’ve memorized Table 8.4 by now and can apply what you’ve learned to Table 8.6.

Türkçe: Functional interface’lerin çoğu `double`, `int` ve `long` içindir. Table 8.6 bu primitive’ler için Table 8.4’ün equivalent’ını gösterir. Bunu ezberlemeniz gerektiğine muhtemelen şaşırmayacaksınız. Neyse ki artık Table 8.4’ü ezberlediniz ve öğrendiklerinizi Table 8.6’ya uygulayabilirsiniz.

Table 8.6 · Common functional interfaces for primitives · Part 1

Functional interfaces / Fonksiyonel arayüzler	Return type / Dönüş tipi	Single abstract method / Tek abstract method	Number of parameters / Parameter sayısı
<code>DoubleSupplier</code>	<code>double</code>	<code>getAsDouble</code>	0
<code>IntSupplier</code>	<code>int</code>	<code>getAsInt</code>	0
<code>LongSupplier</code>	<code>long</code>	<code>getAsLong</code>	0
<code>DoubleConsumer</code>	<code>void</code>	<code>accept</code>	1 (<code>double</code>)
<code>IntConsumer</code>	<code>void</code>	<code>accept</code>	1 (<code>int</code>)
<code>LongConsumer</code>	<code>void</code>	<code>accept</code>	1 (<code>long</code>)
<code>DoublePredicate</code>	<code>boolean</code>	<code>test</code>	1 (<code>double</code>)
<code>IntPredicate</code>	<code>boolean</code>	<code>test</code>	1 (<code>int</code>)
<code>LongPredicate</code>	<code>boolean</code>	<code>test</code>	1 (<code>long</code>)
<code>DoubleFunction<R></code>	<code>R</code>	<code>apply</code>	1 (<code>double</code>)
<code>IntFunction<R></code>	<code>R</code>	<code>apply</code>	1 (<code>int</code>)
<code>LongFunction<R></code>	<code>R</code>	<code>apply</code>	1 (<code>long</code>)

Table 8.6 · Common functional interfaces for primitives · Part 2

Functional interfaces / Fonksiyonel arayüzler	Return type / Dönüş tipi	Single abstract method / Tek abstract method	Number of parameters / Parameter sayısı
<code>DoubleUnaryOperator</code>	<code>double</code>	<code>applyAsDouble</code>	1 (<code>double</code>)
<code>IntUnaryOperator</code>	<code>int</code>	<code>applyAsInt</code>	1 (<code>int</code>)
<code>LongUnaryOperator</code>	<code>long</code>	<code>applyAsLong</code>	1 (<code>long</code>)
<code>DoubleBinaryOperator</code>	<code>double</code>	<code>applyAsDouble</code>	2 (<code>double</code> , <code>double</code>)
<code>IntBinaryOperator</code>	<code>int</code>	<code>applyAsInt</code>	2 (<code>int</code> , <code>int</code>)
<code>LongBinaryOperator</code>	<code>long</code>	<code>applyAsLong</code>	2 (<code>long</code> , <code>long</code>)

English: There are a few things to notice that are different between Table 8.4 and Table 8.6: Generics are gone from some of the interfaces, and instead the type name tells us what primitive type is involved. In other cases, such as `IntFunction`, only the return type generic is needed because we're converting a primitive `int` into an object. The single abstract method is often renamed when a primitive type is returned.

Türkçe: Table 8.4 ile Table 8.6 arasında dikkat edilmesi gereken birkaç fark vardır: Interface'lerin bazılarında generics kalkmıştır; bunun yerine type adı hangi primitive type'ın söz konusu olduğunu bildirir. `IntFunction` gibi diğer durumlarda primitive `int` object'e dönüştürdüğümüz için yalnızca return type generic'ine ihtiyaç vardır. Primitive type döndürüldüğünde single abstract method çoğu zaman yeniden adlandırılır.

English: In addition to Table 8.4 equivalents, some interfaces are specific to primitives. Table 8.7 lists these.

Türkçe: Table 8.4 equivalent'larına ek olarak bazı interface'ler primitive'lere özgüdür. Table 8.7 bunları listeler.

English: We've been using functional interfaces for a while now, so you should have a good grasp of how to read the table. Let's do one example just to be sure. Which functional interface would you use to fill in the blank to make the following code compile?

Türkçe: Bir süredir functional interface'leri kullandığımız için tabloyu nasıl okuyacağınızı iyi kavramış olmalısınız. Emin olmak üzere bir örnek yapalım. Aşağıdaki kodun derlenmesi için boşluğu hangi functional interface ile doldururdunuz?

```
var d = 1.0;
_____ f1 = x -> 1;
f1.applyAsInt(d);
```

English: When you see a question like this, look for clues. You can see that the functional interface in question takes a `double` parameter and returns an `int`. You can also see that it has a single abstract method named `applyAsInt`. The `DoubleToIntFunction` and `ToIntFunction` functional interfaces meet all three of those criteria.

Türkçe: Böyle bir soru gördüğünüzde ipuçlarını arayın. Söz konusu functional interface'in `double` parameter alıp `int` döndürdüğünü görebilirsiniz. Ayrıca `applyAsInt` adlı single abstract method'u olduğunu görürsünüz. `DoubleToIntFunction` ve `ToIntFunction` functional interface'leri bu üç ölçütün tamamını karşılar.

Working with Variables in Lambdas

English: Now that we've learned about functional interfaces, we will use them to show different approaches for variables. They can appear in three places with respect to lambdas: the parameter list, local variables declared inside the lambda body, and variables referenced from the lambda body. All three of these are opportunities for the exam to trick you. We explore each one so you'll be alert when tricks show up!

Türkçe: Functional interface'leri öğrendiğimize göre variable'lara yönelik farklı yaklaşımları göstermek için onları kullanacağız. Variable'lar lambda'larla ilişkili üç yerde görünebilir: parameter list, lambda body'sinde bildirilen local variable'lar ve lambda body'sinden reference verilen variable'lar. Bunların üçü de sınavın sizi yanıltması için fırsattır. Tuzaklar ortaya çıktığında uyanık olmanız için her birini inceliyoruz!

Table 8.7 · Primitive-specific functional interfaces

Functional interfaces / Fonksiyonel arayüzler	Return type / Dönüş tipi	Single abstract method / Tek abstract method	Number of parameters / Parameter sayısı
<code>ToDoubleFunction<T></code>	<code>double</code>	<code>applyAsDouble</code>	1 (T)
<code>ToIntFunction<T></code>	<code>int</code>	<code>applyAsInt</code>	1 (T)
<code>ToLongFunction<T></code>	<code>long</code>	<code>applyAsLong</code>	1 (T)
<code>ToDoubleBiFunction<T, U></code>	<code>double</code>	<code>applyAsDouble</code>	2 (T, U)
<code>ToIntBiFunction<T, U></code>	<code>int</code>	<code>applyAsInt</code>	2 (T, U)
<code>ToLongBiFunction<T, U></code>	<code>long</code>	<code>applyAsLong</code>	2 (T, U)
<code>DoubleToIntFunction</code>	<code>int</code>	<code>applyAsInt</code>	1 (double)
<code>DoubleToLongFunction</code>	<code>long</code>	<code>applyAsLong</code>	1 (double)
<code>IntToDoubleFunction</code>	<code>double</code>	<code>applyAsDouble</code>	1 (int)
<code>IntToLongFunction</code>	<code>long</code>	<code>applyAsLong</code>	1 (int)
<code>LongToDoubleFunction</code>	<code>double</code>	<code>applyAsDouble</code>	1 (long)
<code>LongToIntFunction</code>	<code>int</code>	<code>applyAsInt</code>	1 (long)
<code>ObjDoubleConsumer<T></code>	<code>void</code>	<code>accept</code>	2 (T, double)
<code>ObjIntConsumer<T></code>	<code>void</code>	<code>accept</code>	2 (T, int)
<code>ObjLongConsumer<T></code>	<code>void</code>	<code>accept</code>	2 (T, long)

Listing Parameters

English: Earlier in this chapter, you learned that specifying the type of parameters is optional. Additionally, `var` can be used in place of the specific type. That means that all three of these statements are interchangeable:

Türkçe: Bu bölümün önceki kısmında parameter type'ını belirtmenin optional olduğunu öğrendiniz. Ayrıca belirli type yerine `var` kullanılabilir. Bu, aşağıdaki üç statement'ın birbirinin yerine kullanılabildiği anlamına gelir:

```
Predicate<String> p = x -> true;
Predicate<String> p = (var x) -> true;
Predicate<String> p = (String x) -> true;
```

English: The exam might ask you to identify the type of the lambda parameter. In our example, the answer is `String`. How did we figure that out? A lambda infers the types from the surrounding context. That means you get to do the same.

Türkçe: Sınav sizden lambda parameter'ının type'ını belirlemenizi isteyebilir. Örneğimizde cevap `String`dir. Bunu nasıl bulduk? Lambda type'ları çevreleyen context'ten infer eder. Bu, sizin de aynısını yapmanız gerektiği anlamına gelir.

English: In this case, the lambda is being assigned to a `Predicate` that takes a `String`. Another place to look for the type is in a method signature. Let's try another example. Can you figure out the type of `x`?

Türkçe: Burada lambda, `String` alan bir `Predicate` atanır. Type'ı arayabileceğiniz başka bir yer method signature'dır. Başka bir örnek deneyelim. `x`'in type'ını bulabilir misiniz?

```
public void whatAmI() {
    consume((var x) -> System.out.print(x), 123);
}
```

```
public void consume(Consumer<Integer> c, int num) {
    c.accept(num);
}
```

English: If you guessed `Integer`, you were right. The `whatAmI()` method creates a lambda to be passed to the `consume()` method. Since the `consume()` method expects an `Integer` as the generic, we know that is what the inferred type of `x` will be.

Türkçe: `Integer` tahmin ettiyseniz doğru bildiniz. `whatAmI()` method'u `consume()` method'una geçirilecek bir lambda oluşturur. `consume()` generic olarak `Integer` beklediğinden `x`'in inferred type'ının da bu olacağını biliriz.

English: But wait; there's more. In some cases, you can determine the type without even seeing the method signature. What do you think the type of `x` is here?

Türkçe: Fakat dahası var. Bazı durumlarda method signature'ı görmeden de type'ı belirleyebilirsiniz. Sizce burada `x`'in type'ı nedir?

```
public void counts(List<Integer> list) {
    list.sort((var x, var y) -> x.compareTo(y));
}
```

English: The answer is again `Integer`. Since we are sorting a list, we can use the type of the list to determine the type of the lambda parameter.

Türkçe: Cevap yine `Integer`'dir. Bir listeyi sort ettiğimiz için lambda parameter'ının type'ını belirlemek üzere listenin type'ını kullanabiliriz.

English: Since lambda parameters are just like method parameters, you can add modifiers to them. Specifically, you can add the `final` modifier or an annotation, as shown in this example:

Türkçe: Lambda parameter'ları tıpkı method parameter'ları gibi olduğundan onlara modifier ekleyebilirsiniz. Özellikle bu örnekte gösterildiği gibi `final` modifier veya annotation ekleyebilirsiniz:

```
public void counts(List<Integer> list) {  
    list.sort((final var x, @Deprecated var y) -> x.compareTo(y));  
}
```

English: While this tends to be uncommon in real life, modifiers such as these have been known to appear in passing on the exam.

Türkçe: Gerçek hayatta pek yaygın olmasa da bu tür modifier'ların sınavda dolaylı biçimde karşınıza çıktığı bilinmektedir.

Parameter List Formats

English: You have three formats for specifying parameter types within a lambda: without types, with types, and with `var`. The compiler requires all parameters in the lambda to use the same format. Can you see why the following are not valid?

Türkçe: Lambda içinde parameter type'larını belirtmek için üç formatınız vardır: type'sız, type'lı ve `var` ile. Compiler lambda'daki bütün parameter'ların aynı formatı kullanmasını gerektirir. Aşağıdakilerin neden geçersiz olduğunu görebiliyor musunuz?

```
5: (var x, y) -> "Hello"           // DOES NOT COMPILE  
6: (var x, Integer y) -> true      // DOES NOT COMPILE  
7: (String x, var y, Integer z) -> true // DOES NOT COMPILE  
8: (Integer x, y) -> "goodbye"     // DOES NOT COMPILE
```

English: Line 5 needs to remove `var` from `x` or add it to `y`. Next, lines 6 and 7 need to use the type or `var` consistently. Finally, line 8 needs to remove `Integer` from `x` or add a type to `y`.

Türkçe: Satır 5'in `x`ten `var`ı kaldırması veya `y`ye eklemesi gerekir. Sonra satır 6 ve 7'nin type veya `var`ı tutarlı kullanması gerekir. Son olarak satır 8'in `x`ten `Integer`ı kaldırması veya `y`ye bir type eklemesi gerekir.

Editor note: The source says “Lines 5 needs”; the English above silently corrects this grammatical typo to “Line 5 needs.” **Editör notu:** Kaynaktaki “Lines 5 needs” dil bilgisi hatası İngilizce metinde sessizce “Line 5 needs” olarak düzeltilmiştir.

Using Local Variables Inside a Lambda Body

English: While it is most common for a lambda body to be a single expression, it is legal to define a block. That block can have anything that is valid in a normal Java block, including local variable declarations.

Türkçe: Lambda body'sinin tek bir expression olması en yaygın kullanım olsa da block tanımlamak geçerlidir. Bu block, local variable declaration'lar dahil normal bir Java block'unda geçerli olan her şeyi içerebilir.

English: The following code does just that. It creates a local variable named `c` that is scoped to the lambda block:

Türkçe: Aşağıdaki kod tam olarak bunu yapar. Scope'u lambda block'u olan `c` adlı local variable oluşturur:

```
(a, b) -> { int c = 0; return 5; }
```

English: Now let's try another one. Do you see what's wrong here?

Türkçe: Şimdi başka bir tane deneyelim. Burada neyin yanlış olduğunu görüyor musunuz?

```
(a, b) -> { int a = 0; return 5; } // DOES NOT COMPILE
```

English: We tried to redeclare `a`, which is not allowed. Java doesn't let you create a local variable with the same name as one already declared in that scope. While this kind of error is less likely to come up in real life, it has been known to appear on the exam!

Türkçe: İzin verilmeyen şekilde `a`'yı yeniden bildirmeye çalıştık. Java, aynı scope'ta zaten bildirilmiş bir variable ile aynı adda local variable oluşturmanıza izin vermez. Bu tür bir hatayla gerçek hayatta karşılaşma olasılığı düşük olsa da sınavda görüldüğü bilinmektedir!

English: Now let's try a hard one. How many syntax errors do you see in this method?

Türkçe: Şimdi zor bir tane deneyelim. Bu method'da kaç syntax error görüyorsunuz?

```
11: public void variables(int a) {  
12:     int b = 1;  
13:     Predicate<Integer> p1 = a -> {  
14:         int b = 0;  
15:         int c = 0;  
16:         return b == c; }  
17: }
```

English: There are three syntax errors. The first is on line 13. The variable `a` was already used in this scope as a method parameter, so it cannot be reused. The next syntax error comes on line 14, where the code attempts to redeclare local variable `b`. The third syntax error is quite subtle and on line 16. See it? Look really closely.

Türkçe: Üç syntax error vardır. İlki satır 13'tedir. `a` variable'ı bu scope'ta method parameter olarak zaten kullanıldığından yeniden kullanılamaz. Sonraki syntax error, kodun local variable `b`'yi yeniden bildirmeye çalıştığı satır 14'tedir. Üçüncü syntax error oldukça ince ve satır 16'dadır. Gördünüz mü? Çok dikkatli bakın.

English: The variable `p1` is missing a semicolon at the end. There is a semicolon before the `}`, but that is inside the block. While you don't normally have to look for missing semicolons, lambdas are tricky in this space, so beware!

Türkçe: Variable `p1`'in sonunda semicolon eksiktir. `}` öncesinde bir semicolon vardır; ancak o block'un içindedir. Normalde eksik semicolon aramanız gerekmez de lambda'lar bu konuda yanıltıcıdır; dikkatli olun!

Keep Your Lambdas Short

English: Having a lambda with multiple lines and a `return` statement is often a clue that you should refactor and put that code in a method. For example, the previous example could be rewritten as

Türkçe: Birden çok satırı ve `return` statement'ı bulunan lambda çoğu zaman refactor yapıp kodu bir method'a taşımanız gerektiğinin ipucudur. Örneğin önceki örnek şöyle yeniden yazılabilir:

```
Predicate<Integer> p1 = a -> returnSame(a);
```

Java 17 düzeltme notu: Kaynağın hemen önceki literal scope'unda method parameter'ın adı da `a`dır (`variables(int a)`). Lambda parameter aynı scope'taki method parameter'ı redeclare edemeyeceğinden bu satır o bağlamda yine **Does not compile** sonucunu verir. Geçerli refactor, farklı bir lambda parameter adı kullanmalıdır:

```
Predicate<Integer> p1 = x -> returnSame(x);
```

English: This simpler form can be further refactored to use a method reference:

Türkçe: Bu daha basit biçim, method reference kullanacak şekilde daha da refactor edilebilir:

```
Predicate<Integer> p1 = this::returnSame;
```

English: You might be wondering why this is so important. In Chapter 10, lambdas and method references are used in chained method calls. The shorter the lambda, the easier it is to read the code.

Türkçe: Bunun neden bu kadar önemli olduğunu merak ediyordunuz. Chapter 10'da lambda'lar ve method reference'lar chained method call'larda kullanılır. Lambda ne kadar kısa olursa kodu okumak o kadar kolay olur.

Referencing Variables from the Lambda Body

English: Lambda bodies are allowed to reference some variables from the surrounding code. The following code is legal:

Türkçe: Lambda body'lerinin çevreleyen koddaki bazı variable'lara reference vermesine izin verilir. Aşağıdaki kod geçerlidir:

```
public class Crow {
    private String color;
    public void caw(String name) {
        String volume = "loudly";
        Consumer<String> consumer = s ->
            System.out.println(name + " says "
                + volume + " that she is " + color);
    }
}
```

English: This shows that a lambda can access an instance variable, method parameter, or local variable under certain conditions. Instance variables (and class variables) are always allowed. The only thing lambdas cannot access are variables that are not final or effectively final. If you need a refresher on effectively final, see Chapter 5, “Methods.”

Türkçe: Bu, lambda'nın belirli koşullar altında instance variable, method parameter veya local variable'a erişebildiğini gösterir. Instance variable'lara (ve class variable'lara) her zaman izin verilir. Lambda'ların erişemediği variable'lar, **final** veya effectively final olmayanlardır. Effectively final konusunda hatırlatmaya ihtiyacınız varsa Chapter 5, “Methods” bölümüne bakın.

English: It gets even more interesting when you look at where the compiler errors occur when the variables are not effectively final.

Türkçe: Variable'lar effectively final olmadığında compiler error'ların nerede oluştuğuna baktığınızda konu daha da ilginç hâle gelir.

```
2: public class Crow {
3:     private String color;
4:     public void caw(String name) {
5:         String volume = "loudly";
6:         name = "Catty";
7:         color = "black";
8:
9:         Consumer<String> consumer = s ->
10:             System.out.println(name + " says "           // DOES NOT COMPILE
11:                 + volume + " that she is " + color); // DOES NOT COMPILE
12:         volume = "softly";
13:     }
14: }
```

English: In this example, the method parameter **name** is not effectively final because it is set on line 6. However, the compiler error occurs on line 10. It's not a problem to assign a value to a non-final variable. However, once the lambda tries to use it, we do have a problem. The variable is no longer effectively final, so the lambda is not allowed to use the variable.

Türkçe: Bu örnekte method parameter **name**, satır 6'da değer atandığı için effectively final değildir. Buna karşın compiler error satır 10'da oluşur. Non-final bir variable'a değer atamak tek başına sorun değildir. Ancak lambda onu kullanmaya çalışınca sorun oluşur. Variable artık effectively final olmadığından lambda'nın onu kullanmasına izin verilmez.

English: The variable `volume` is not effectively final either since it is updated on line 12. In this case, the compiler error is on line 11. That's before the reassignment! Again, the act of assigning a value is only a problem from the point of view of the lambda. Therefore, the lambda has to be the one to generate the compiler error.

Türkçe: `volume` variable'ı da satır 12'de update edildiği için effectively final değildir. Bu durumda compiler error satır 11'dedir; yani reassignment'tan öncedir! Yine bir değer atama işlemi yalnızca lambda'nın bakış açısından sorundur. Bu nedenle compiler error'ı üretmesi gereken yer lambda'dır.

English: To review, make sure you've memorized Table 8.8.

Türkçe: Tekrar amacıyla Table 8.8'i ezberlediğinizden emin olun.

Table 8.8 · Rules for accessing a variable from a lambda body inside a method

Variable type / Variable türü	Rule / Kural
Instance variable	Allowed / İzin verilir
Static variable	Allowed / İzin verilir
Local variable	Allowed if final or effectively final / <code>final</code> veya effectively final ise izin verilir
Method parameter	Allowed if final or effectively final / <code>final</code> veya effectively final ise izin verilir
Lambda parameter	Allowed / İzin verilir

Summary / Özet

English: We spent a lot of time in this chapter teaching you how to use lambda expressions, and with good reason. The next two chapters depend heavily on your ability to create and use lambda expressions. We recommend that you understand this chapter well before moving on.

Türkçe: Bu bölümde lambda expression'ların nasıl kullanılacağını öğretmeye çok zaman ayırdık ve bunun iyi bir nedeni vardır. Sonraki iki bölüm, lambda expression oluşturup kullanabilme yeteneğinize büyük ölçüde bağlıdır. Devam etmeden önce bu bölümü iyi anlamanızı öneririz.

English: Lambda expressions, or lambdas, allow passing around blocks of code. The full syntax looks like this:

Türkçe: Lambda expression'lar veya kısaca lambda'lar, kod bloklarının bir yerden başka bir yere geçirilmesine olanak tanır. Tam syntax şöyle görünür:

```
(String a, String b) -> { return a.equals(b); }
```

English: The parameter types can be omitted. When only one parameter is specified without a type, the parentheses can also be omitted. The braces and `return` statement can be omitted for a single statement, making the short form as follows:

Türkçe: Parameter type'ları atlanabilir. Type'ı olmayan yalnızca bir parameter belirtildiğinde parentheses de atlanabilir. Tek statement için braces ve `return` statement atlanarak şu kısa biçim elde edilebilir:

```
a -> a.equals(b)
```

English: Lambdas can be passed to a method expecting an instance of a functional interface. A lambda can define parameters or variables in the body as long as their names are different from existing local variables. The body of a lambda is allowed to use any instance or class variables.

Türkçe: Lambda'lar functional interface instance'ı bekleyen bir method'a geçirilebilir. Lambda, adları mevcut local variable'lardan farklı olduğu sürece body'de parameter veya variable tanımlayabilir. Lambda body'sinin herhangi bir instance veya class variable'ı kullanmasına izin verilir.

English: Additionally, it can use any local variables or method parameters that are final or effectively final.

Türkçe: Ayrıca `final` veya effectively final olan herhangi bir local variable'ı ya da method parameter'ı kullanabilir.

English: A method reference is a compact syntax for writing lambdas that refer to methods. There are four types: static methods, instance methods on a particular object, instance methods on a parameter, and constructor references.

Türkçe: Method reference, method'lara reference veren lambda'ları yazmak için compact bir syntax'tır. Dört türü vardır: static method'lar, belirli bir object üzerindeki instance method'lar, bir parameter üzerindeki instance method'lar ve constructor reference'lar.

English: A functional interface has a single abstract method. Any functional interface can be implemented with a lambda expression. You must know the built-in functional interfaces.

Türkçe: Functional interface'in tek bir abstract method'u vardır. Herhangi bir functional interface lambda expression ile implement edilebilir. Built-in functional interface'leri bilmelisiniz.

English: You should review the tables in the chapter. While there are many tables, some share common patterns, making it easier to remember them. You absolutely must memorize Table 8.4, which lists the common functional interfaces.

Türkçe: Bölümdeki tabloları gözden geçirmelisiniz. Çok sayıda tablo olsa da bazıları ortak pattern'ları paylaşarak hatırlanmalarını kolaylaştırır. Yaygın functional interface'leri listeleyen Table 8.4'ü mutlaka ezberlemelisiniz.

Exam Essentials / Sınav İçin Temel Noktalar

English: Write simple lambda expressions. Look for the presence or absence of optional elements in lambda code. Parameter types are optional. Braces and the `return` keyword are optional when the body is a single statement. Parentheses are optional when only one parameter is specified and the type is implicit.

Türkçe: Basit lambda expression'lar yazın. Lambda kodunda optional öğelerin bulunup bulunmadığına bakın. Parameter type'ları optional'dır. Body tek statement olduğunda braces ve `return` keyword'ü optional'dır. Yalnızca bir parameter belirtilip type implicit olduğunda parentheses optional'dır.

English: Determine whether a variable can be used in a lambda body. Local variables and method parameters must be final or effectively final to be referenced. This means the code must compile if you were to add the `final` keyword to these variables. Instance and class variables are always allowed.

Türkçe: Bir variable'ın lambda body'sinde kullanılıp kullanılmayacağını belirleyin. Local variable ve method parameter'lara reference verilebilmesi için bunlar `final` veya effectively final olmalıdır. Bu, söz konusu variable'lara `final` keyword'ünü ekleyiniz kodun derlenmesi gerektiği anlamına gelir. Instance ve class variable'lara her zaman izin verilir.

English: Translate method references to the “long form” lambda. Be able to convert method references into regular lambda expressions and vice versa. For example, `System.out::print` and `x -> System.out.print(x)` are equivalent. Remember that the order of method parameters is inferred when using a method reference.

Türkçe: Method reference'ları “long form” lambda'ya çevirin. Method reference'ları normal lambda expression'lara ve tersine dönüştürebilir. Örneğin `System.out::print` ile `x -> System.out.print(x)` equivalent'tır. Method reference kullanırken method parameter sırasının infer edildiğini unutmayın.

English: Determine whether an interface is a functional interface. Use the single abstract method (SAM) rule to determine whether an interface is a functional interface. Other interface method types (`default`, `private`, `static`, and `private static`) do not count toward the single abstract method count, nor do any public methods with signatures found in `Object`.

Türkçe: Bir interface'in functional interface olup olmadığını belirleyin. Bir interface'in functional interface olup olmadığını belirlemek için single abstract method (SAM) kuralını kullanın. Diğer interface method türleri (`default`, `private`, `static` ve `private static`) single abstract method sayısına katılmaz; `Object`'te bulunan signature'lara sahip public method'lar da katılmaz.

English: Identify the correct functional interface given the number of parameters, return type, and method name—and vice versa. The most common functional interfaces are `Supplier`, `Consumer`, `Function`, and `Predicate`. There are also binary versions and primitive versions of many of these methods. You can use the number of parameters and return type to tell them apart.

Türkçe: Parameter sayısı, return type ve method adı verildiğinde doğru functional interface'i—ve tersini—belirleyin. En yaygın functional interface'ler `Supplier`, `Consumer`, `Function` ve `Predicate`'dir. Bu method'ların birçoğunun binary ve primitive sürümleri de vardır. Bunları birbirinden ayırmak için parameter sayısını ve return type'ı kullanabilirsiniz.

Review Questions / Gözden Geçirme Soruları

English: The answers to the chapter review questions can be found in the Appendix.

Türkçe: Chapter review questions'ın cevapları Appendix'te bulunabilir.

Question 1 / Soru 1

English: What is the result of the following class?

Türkçe: Aşağıdaki class'ın sonucu nedir?

```
1: import java.util.function.*;
2:
3: public class Panda {
4:     int age;
5:     public static void main(String[] args) {
6:         Panda p1 = new Panda();
7:         p1.age = 1;
8:         check(p1, p -> p.age < 5);
9:     }
10:    private static void check(Panda panda,
11:        Predicate<Panda> pred) {
12:        String result =
13:            pred.test(panda) ? "match" : "not match";
14:        System.out.print(result);
15:    } }
```

English: A. `match`

Türkçe: A. `match`

English: B. `not match`

Türkçe: B. `not match`

English: C. Compiler error on line 8

Türkçe: C. Satır 8'de compiler error

English: D. Compiler error on lines 10 and 11

Türkçe: D. Satır 10 ve 11'de compiler error

English: E. Compiler error on lines 12 and 13

Türkçe: E. Satır 12 ve 13'te compiler error

English: F. A runtime exception is thrown.

Türkçe: F. Runtime'da exception fırlatılır.

Question 2 / Soru 2

English: What is the result of the following code?

Türkçe: Aşağıdaki kodun sonucu nedir?

```
1: interface Climb {
2:     boolean isTooHigh(int height, int limit);
3: }
4:
5: public class Climber {
6:     public static void main(String[] args) {
7:         check(h, m) -> h.append(m).isEmpty(), 5);
8:     }
9:     private static void check(Climb climb, int height) {
10:         if (climb.isTooHigh(height, 10))
11:             System.out.println("too high");
12:         else
13:             System.out.println("ok");
14:     }
15: }
```

English: A. `ok`

Türkçe: A. `ok`

English: B. `too high`

Türkçe: B. `too high`

English: C. Compiler error on line 7

Türkçe: C. Satır 7'de compiler error

English: D. Compiler error on line 10

Türkçe: D. Satır 10'da compiler error

English: E. Compiler error on a different line

Türkçe: E. Başka bir satırda compiler error

English: F. A runtime exception is thrown.

Türkçe: F. Runtime'da exception fırlatılır.

Question 3 / Soru 3

English: Which statements about functional interfaces are true? (Choose all that apply.)

Türkçe: Functional interface'lerle ilgili hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

English: A. A functional interface can contain default and private methods.

Türkçe: A. Functional interface default ve private method'lar içerebilir.

English: B. A functional interface can be defined as a class or an interface.

Türkçe: B. Functional interface bir class veya interface olarak tanımlanabilir.

English: C. Abstract methods with signatures that are contained in public methods of `java.lang.Object` do not count toward the abstract method count for a functional interface.

Türkçe: C. `java.lang.Object`'in public method'larında bulunan signature'lara sahip abstract method'lar, functional interface'in abstract method sayısına katılmaz.

English: D. A functional interface cannot contain static or private static methods.

Türkçe: D. Functional interface static veya private static method içeremez.

English: E. A functional interface must be marked with the `@FunctionalInterface` annotation.

Türkçe: E. Functional interface `@FunctionalInterface` annotation'ıyla işaretlenmek zorundadır.

Question 4 / Soru 4

English: Which lambda can replace the `MySecret` class to return the same value? (Choose all that apply.)

Türkçe: Hangi lambda aynı değeri döndürmek üzere `MySecret` class'ının yerini alabilir? (Uygun olanların tümünü seçin.)

```
interface Secret {
    String magic(double d);
}
class MySecret implements Secret {
    public String magic(double d) {
        return "Poof";
    }
}
```

English: A. (e) -> "Poof"

Türkçe: A. (e) -> "Poof"

English: B. (e) -> {"Poof"}

Türkçe: B. (e) -> {"Poof"}

English: C. (e) -> { String e = ""; "Poof" }

Türkçe: C. (e) -> { String e = ""; "Poof" }

English: D. (e) -> { String e = ""; return "Poof"; }

Türkçe: D. (e) -> { String e = ""; return "Poof"; }

English: E. (e) -> { String e = ""; return "Poof" }

Türkçe: E. (e) -> { String e = ""; return "Poof" }

English: F. (e) -> { String f = ""; return "Poof"; }

Türkçe: F. (e) -> { String f = ""; return "Poof"; }

Question 5 / Soru 5

English: Which of the following functional interfaces contain an abstract method that returns a primitive value? (Choose all that apply.)

Türkçe: Aşağıdaki functional interface'lerden hangileri primitive value döndüren bir abstract method içerir? (Uygun olanların tümünü seçin.)

English: A. `BooleanSupplier`

Türkçe: A. `BooleanSupplier`

English: B. `CharSupplier`

Türkçe: B. `CharSupplier`

English: C. `DoubleSupplier`

Türkçe: C. `DoubleSupplier`

English: D. `FloatSupplier`

Türkçe: D. `FloatSupplier`

English: E. `IntSupplier`

Türkçe: E. `IntSupplier`

English: F. `StringSupplier`

Türkçe: F. `StringSupplier`

Question 6 / Soru 6

English: Which of the following lambda expressions can be passed to a function of `Predicate<String>` type? (Choose all that apply.)

Türkçe: Aşağıdaki lambda expression'lardan hangileri `Predicate<String>` type'ındaki bir function'a geçirilebilir? (Uygun olanların tümünü seçin.)

English: A. `s -> s.isEmpty()`

Türkçe: A. `s -> s.isEmpty()`

English: B. `s --> s.isEmpty()`

Türkçe: B. `s --> s.isEmpty()`

English: C. `(String s) -> s.isEmpty()`

Türkçe: C. `(String s) -> s.isEmpty()`

English: D. `(String s) --> s.isEmpty()`

Türkçe: D. `(String s) --> s.isEmpty()`

English: E. `(StringBuilder s) -> s.isEmpty()`

Türkçe: E. `(StringBuilder s) -> s.isEmpty()`

English: F. `(StringBuilder s) --> s.isEmpty()`

Türkçe: F. `(StringBuilder s) --> s.isEmpty()`

Question 7 / Soru 7

English: Which of these statements is true about the following code?

Türkçe: Aşağıdaki kodla ilgili bu ifadelerden hangisi doğrudur?

```
public void method() {
    x((var x) -> {}, (var x, var y) -> false);
}
public void x(Consumer<String> x, BinaryOperator<Boolean> y) {}
```

English: A. The code does not compile because of one of the variables named `x`.

Türkçe: A. `x` adlı variable'lardan biri nedeniyle kod derlenmez.

English: B. The code does not compile because of one of the variables named `y`.

Türkçe: B. `y` adlı variable'lardan biri nedeniyle kod derlenmez.

English: C. The code does not compile for another reason.

Türkçe: C. Kod başka bir nedenle derlenmez.

English: D. The code compiles, and the `x` in each lambda refers to the same type.

Türkçe: D. Kod derlenir ve her lambda'daki `x` aynı type'a gönderme yapar.

English: E. The code compiles, and the `x` in each lambda refers to a different type.

Türkçe: E. Kod derlenir ve her lambda'daki `x` farklı bir type'a gönderme yapar.

Question 8 / Soru 8

English: Which of the following is equivalent to this code? (Choose all that apply.)

Türkçe: Aşağıdakilerden hangisi bu koda equivalent'tır? (Uygun olanların tümünü seçin.)

```
UnaryOperator<Integer> u = x -> x * x;
```

English: A. `BiFunction<Integer> f = x -> x*x;`

Türkçe: A. `BiFunction<Integer> f = x -> x*x;`

English: B. `BiFunction<Integer, Integer> f = x -> x*x;`

Türkçe: B. `BiFunction<Integer, Integer> f = x -> x*x;`

English: C. `BinaryOperator<Integer, Integer> f = x -> x*x;`
Türkçe: C. `BinaryOperator<Integer, Integer> f = x -> x*x;`

English: D. `Function<Integer> f = x -> x*x;`
Türkçe: D. `Function<Integer> f = x -> x*x;`

English: E. `Function<Integer, Integer> f = x -> x*x;`
Türkçe: E. `Function<Integer, Integer> f = x -> x*x;`

English: F. None of the above
Türkçe: F. Yukarıdakilerin hiçbirisi

Question 9 / Soru 9

English: Which statements are true? (Choose all that apply.)
Türkçe: Hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

English: A. The `Consumer` interface is good for printing out an existing value.
Türkçe: A. `Consumer` interface'i mevcut bir değeri yazdırmak için uygundur.

English: B. The `Supplier` interface is good for printing out an existing value.
Türkçe: B. `Supplier` interface'i mevcut bir değeri yazdırmak için uygundur.

English: C. The `IntegerSupplier` interface returns an `int`.
Türkçe: C. `IntegerSupplier` interface'i `int` döndürür.

English: D. The `Predicate` interface returns an `int`.
Türkçe: D. `Predicate` interface'i `int` döndürür.

English: E. The `Function` interface has a method named `test()`.
Türkçe: E. `Function` interface'inin `test()` adlı bir method'u vardır.

English: F. The `Predicate` interface has a method named `test()`.
Türkçe: F. `Predicate` interface'inin `test()` adlı bir method'u vardır.

Question 10 / Soru 10

English: Which of the following can be inserted without causing a compilation error? (Choose all that apply.)
Türkçe: Aşağıdakilerden hangileri compilation error'a yol açmadan eklenebilir? (Uygun olanların tümünü seçin.)

```
public void remove(List<Character> chars) {  
    char end = 'z';  
    Predicate<Character> predicate = c -> {  
        char start = 'a'; return start <= c && c <= end; };  
    // INSERT LINE HERE  
}
```

English: A. `char start = 'a';`

Türkçe: A. `char start = 'a';`

English: B. `char c = 'x';`

Türkçe: B. `char c = 'x';`

English: C. `chars = null;`

Türkçe: C. `chars = null;`

English: D. `end = '1';`

Türkçe: D. `end = '1';`

English: E. None of the above

Türkçe: E. Yukarıdakilerin hiçbiri

Question 11 / Soru 11

English: How many times is `true` printed out by this code?

Türkçe: Bu kod `true` değerini kaç kez yazdırır?

```
import java.util.function.Predicate;  
public class Fantasy {  
    public static void scary(String animal) {  
        var dino = s -> "dino".equals(animal);  
        var dragon = s -> "dragon".equals(animal);  
        var combined = dino.or(dragon);  
        System.out.println(combined.test(animal));  
    }  
    public static void main(String[] args) {  
        scary("dino");  
        scary("dragon");  
        scary("unicorn");  
    }  
}
```

English: A. One

Türkçe: A. Bir

English: B. Two

Türkçe: B. İki

English: C. Three

Türkçe: C. Üç

English: D. The code does not compile.

Türkçe: D. Kod derlenmez.

English: E. A runtime exception is thrown.

Türkçe: E. Runtime'da exception fırlatılır.

Question 12 / Soru 12

English: What does the following code output?

Türkçe: Aşağıdaki kod ne yazdırır?

```
Function<Integer, Integer> s = a -> a + 4;  
Function<Integer, Integer> t = a -> a * 3;  
Function<Integer, Integer> c = s.compose(t);  
System.out.print(c.apply(1));
```

English: A. 7

Türkçe: A. 7

English: B. 15

Türkçe: B. 15

English: C. The code does not compile because of the data types in the lambda expressions.

Türkçe: C. Lambda expression'lardaki data type'lar nedeniyle kod derlenmez.

English: D. The code does not compile because of the `compose()` call.

Türkçe: D. `compose()` çağırısı nedeniyle kod derlenmez.

English: E. The code does not compile for another reason.

Türkçe: E. Kod başka bir nedenle derlenmez.

Question 13 / Soru 13

English: Which is true of the following code?

Türkçe: Aşağıdaki kodla ilgili hangisi doğrudur?

```
int length = 3;
for (int i = 0; i < 3; i++) {
    if (i % 2 == 0) {
        Supplier<Integer> supplier = () -> length; // A
        System.out.println(supplier.get());        // B
    } else {
        int j = i;
        Supplier<Integer> supplier = () -> j;      // C
        System.out.println(supplier.get());        // D
    }
}
```

English: A. The first compiler error is on line A.

Türkçe: A. İlk compiler error A satırındadır.

English: B. The first compiler error is on line B.

Türkçe: B. İlk compiler error B satırındadır.

English: C. The first compiler error is on line C.

Türkçe: C. İlk compiler error C satırındadır.

English: D. The first compiler error is on line D.

Türkçe: D. İlk compiler error D satırındadır.

English: E. The code compiles successfully.

Türkçe: E. Kod başarıyla derlenir.

Question 14 / Soru 14

English: Which of the following are valid lambda expressions? (Choose all that apply.)

Türkçe: Aşağıdakilerden hangileri geçerli lambda expression'lardır? (Uygun olanların tümünü seçin.)

English: A. `(Wolf w, var c) -> 39`

Türkçe: A. `(Wolf w, var c) -> 39`

English: B. `(final Camel c) -> {}`

Türkçe: B. `(final Camel c) -> {}`

English: C. `(a,b,c) -> {int b = 3; return 2;}`

Türkçe: C. `(a,b,c) -> {int b = 3; return 2;}`

English: D. `(x,y) -> new RuntimeException()`

Türkçe: D. `(x,y) -> new RuntimeException()`

English: E. (var y) -> return 0;

Türkçe: E. (var y) -> return 0;

English: F. () -> {float r}

Türkçe: F. () -> {float r}

English: G. (Cat a, b) -> {}

Türkçe: G. (Cat a, b) -> {}

Question 15 / Soru 15

English: Which lambda expression, when entered into the blank line in the following code, causes the program to print **hahaha**? (Choose all that apply.)

Türkçe: Aşağıdaki kodun boş satırına girildiğinde hangi lambda expression programın **hahaha** yazdırmasını sağlar? (Uygun olanların tümünü seçin.)

```
import java.util.function.Predicate;
public class Hyena {
    private int age = 1;
    public static void main(String[] args) {
        var p = new Hyena();
        double height = 10;
        int age = 1;
        testLaugh(p, _____);
        age = 2;
    }
    static void testLaugh(Hyena panda, Predicate<Hyena> joke) {
        var r = joke.test(panda) ? "hahaha" : "silence";
        System.out.print(r);
    }
}
```

English: A. var -> p.age <= 10

Türkçe: A. var -> p.age <= 10

English: B. shenzi -> age==1

Türkçe: B. shenzi -> age==1

English: C. p -> true

Türkçe: C. p -> true

English: D. age==1

Türkçe: D. age==1

English: E. shenzi -> age==2

Türkçe: E. shenzi -> age==2

English: F. `h -> h.age < 5`

Türkçe: F. `h -> h.age < 5`

English: G. None of the above, as the code does not compile

Türkçe: G. Yukarıdakilerin hiçbirisi; çünkü kod derlenmez.

Question 16 / Soru 16

English: Which of the following can be inserted without causing a compilation error? (Choose all that apply.)

Türkçe: Aşağıdakilerden hangileri compilation error'a yol açmadan eklenebilir? (Uygun olanların tümünü seçin.)

```
public void remove(List<Character> chars) {  
    char end = 'z';  
    // INSERT LINE HERE  
    Predicate<Character> predicate = c -> {  
        char start = 'a'; return start <= c && c <= end; };  
}
```

English: A. `char start = 'a';`

Türkçe: A. `char start = 'a';`

English: B. `char c = 'x';`

Türkçe: B. `char c = 'x';`

English: C. `chars = null;`

Türkçe: C. `chars = null;`

English: D. `end = '1';`

Türkçe: D. `end = '1';`

English: E. None of the above

Türkçe: E. Yukarıdakilerin hiçbirisi

Question 17 / Soru 17

English: What is the result of running the following class?

Türkçe: Aşağıdaki class çalıştırıldığında sonuç nedir?

```

1: import java.util.function.*;
2:
3: public class Panda {
4:     int age;
5:     public static void main(String[] args) {
6:         Panda p1 = new Panda();
7:         p1.age = 1;
8:         check(p1, p -> {p.age < 5});
9:     }
10:    private static void check(Panda panda,
11:        Predicate<Panda> pred) {
12:        String result = pred.test(panda)
13:            ? "match" : "not match";
14:        System.out.print(result);
15:    } }

```

English: A. `match`

Türkçe: A. `match`

English: B. `not match`

Türkçe: B. `not match`

English: C. Compiler error on line 8

Türkçe: C. Satır 8'de compiler error

English: D. Compiler error on line 10

Türkçe: D. Satır 10'da compiler error

English: E. Compiler error on line 12

Türkçe: E. Satır 12'de compiler error

English: F. A runtime exception is thrown.

Türkçe: F. Runtime'da exception fırlatılır.

Question 18 / Soru 18

English: Which functional interfaces complete the following code? For line 7, assume `m` and `n` are instances of functional interfaces that exist and have the same type as `y`. (Choose three.)

Türkçe: Aşağıdaki kodu hangi functional interface'ler tamamlar? Satır 7 için `m` ve `n`'nin var olan ve `y` ile aynı type'taki functional interface instance'ları olduğunu varsayın. (Üç tane seçin.)

```

6: _____ x = String::new;
7: _____ y = m.andThen(n);
8: _____ z = a -> a + a;

```

English: A. `BinaryConsumer<String, String>`

Türkçe: A. `BinaryConsumer<String, String>`

English: B. BiConsumer<String, String>
Türkçe: B. BiConsumer<String, String>

English: C. BinaryFunction<String, String>
Türkçe: C. BinaryFunction<String, String>

English: D. BiFunction<String, String>
Türkçe: D. BiFunction<String, String>

English: E. Predicate<String>
Türkçe: E. Predicate<String>

English: F. Supplier<String>
Türkçe: F. Supplier<String>

English: G. UnaryOperator<String>
Türkçe: G. UnaryOperator<String>

English: H. UnaryOperator<String, String>
Türkçe: H. UnaryOperator<String, String>

Question 19 / Soru 19

English: Which of the following compiles and prints out the entire set? (Choose all that apply.)
Türkçe: Aşağıdakilerden hangileri derlenir ve set'in tamamını yazdırır? (Uygun olanların tümünü seçin.)

```
Set<?> set = Set.of("lion", "tiger", "bear");  
var s = Set.copyOf(set);  
Consumer<Object> consumer = _____;  
s.forEach(consumer);
```

English: A. () -> System.out.println(s)
Türkçe: A. () -> System.out.println(s)

English: B. s -> System.out.println(s)
Türkçe: B. s -> System.out.println(s)

English: C. (s) -> System.out.println(s)
Türkçe: C. (s) -> System.out.println(s)

English: D. System.out.println(s)
Türkçe: D. System.out.println(s)

English: E. `System.out.println`

Türkçe: E. `System.out.println`

English: F. `System.out.println`

Türkçe: F. `System.out.println`

Question 20 / Soru 20

English: Which lambdas can replace the `new Sloth()` call in the `main()` method and produce the same output at runtime? (Choose all that apply.)

Türkçe: Hangi lambda'lar `main()` method'undaki `new Sloth()` çağrısının yerini alıp runtime'da aynı output'u üretebilir? (Uygun olanların tümünü seçin.)

```
import java.util.List;
interface Yawn {
    String yawn(double d, List<Integer> time);
}
class Sloth implements Yawn {
    public String yawn(double zzz, List<Integer> time) {
        return "Sleep: " + zzz;
    }
}
public class Vet {
    public static String takeNap(Yawn y) {
        return y.yawn(10, null);
    }
    public static void main(String... unused) {
        System.out.print(takeNap(new Sloth()));
    }
}
```

English: A. `(z,f) -> { String x = ""; return "Sleep: " + x }`

Türkçe: A. `(z,f) -> { String x = ""; return "Sleep: " + x }`

English: B. `(t,s) -> { String t = ""; return "Sleep: " + t; }`

Türkçe: B. `(t,s) -> { String t = ""; return "Sleep: " + t; }`

English: C. `(w,q) -> {"Sleep: " + w}`

Türkçe: C. `(w,q) -> {"Sleep: " + w}`

English: D. `(e,u) -> { String g = ""; "Sleep: " + e }`

Türkçe: D. `(e,u) -> { String g = ""; "Sleep: " + e }`

English: E. `(a,b) -> "Sleep: " + (double)(b==null ? a : a)`

Türkçe: E. `(a,b) -> "Sleep: " + (double)(b==null ? a : a)`

English: F. `(r,k) -> { String g = ""; return "Sleep:"; }`

Türkçe: F. `(r,k) -> { String g = ""; return "Sleep:"; }`

English: G. None of the above, as the program does not compile

Türkçe: G. Yukarıdakilerin hiçbirini; çünkü program derlenmez.

Question 21 / Soru 21

English: Which of the following are valid functional interfaces? (Choose all that apply.)

Türkçe: Aşağıdakilerden hangileri geçerli functional interface'tir? (Uygun olanların tümünü seçin.)

```
public interface Transport {
    public int go();
    public boolean equals(Object o);
}

public abstract class Car {
    public abstract Object swim(double speed, int duration);
}

public interface Locomotive extends Train {
    public int getSpeed();
}

public interface Train extends Transport {}

abstract interface Spaceship extends Transport {
    default int blastOff();
}

public interface Boat {
    int hashCode();
    int hashCode(String input);
}
```

English: A. Boat

Türkçe: A. Boat

English: B. Car

Türkçe: B. Car

English: C. Locomotive

Türkçe: C. Locomotive

English: D. Spaceship

Türkçe: D. Spaceship

English: E. Transport

Türkçe: E. Transport

English: F. Train

Türkçe: F. Train

English: G. None of these is a valid functional interface.

Türkçe: G. Bunların hiçbirisi geçerli functional interface değildir.

Kaynak kapsam notu: PDF sayfası 462 boştur; 419–462 fiziksel kaynak aralığının eksiksiz izlendiğini göstermek için kaynak sayfa işareti korunmuştur.

Appendix · Official Review Question Answers / Resmî Cevaplar

Bu bölüm, kaynağın **Answers to the Review Questions** ekindeki Chapter 8 cevaplarını PDF sayfaları 936–939'dan aktarır. Cevaplar, soruların okuma akışına sızması için bütün Question 1–21 metinlerinden sonra tutulur. Aşağıdaki cevap kaynağı sayfa işaretleri, ana bölümün 419–462 sayfalarına ait kapsam doğrulamasından bağımsızdır.

Official Answer 1

English: A. This code is correct. Line 8 creates a lambda expression that checks whether the age is less than 5, making option A correct. Since there is only one parameter and it does not specify a type, the parentheses around the parameter are optional. Lines 11 and 13 use the `Predicate` interface, which declares a `test()` method.

Türkçe: A. Bu kod doğrudur. Satır 8, age değerinin 5'ten küçük olup olmadığını kontrol eden bir lambda expression oluşturur; dolayısıyla A seçeneği doğrudur. Yalnızca bir parameter bulunduğu ve type belirtilmediği için parameter'ın çevresindeki parentheses optional'dır. Satır 11 ve 13, `test()` method'unu bildiren `Predicate` interface'ini kullanır.

Official Answer 2

English: C. The interface takes two `int` parameters. The code on line 7 attempts to use them as if `h` is a `String`, making option C correct. It is tricky to use types in a lambda when they are implicitly specified. Remember to check the interface for the real type.

Türkçe: C. Interface iki `int` parameter alır. Satır 7'deki kod, `h`yi `String`miş gibi kullanmaya çalışır; dolayısıyla C seçeneği doğrudur. Type'lar implicit belirtildiğinde lambda içindeki kullanımı yanıltıcı olabilir. Gerçek type'ı bulmak için interface'i kontrol etmeyi unutmayın.

Official Answer 3

English: A, C. A functional interface can contain any number of non-abstract methods, including `default`, `private`, `static`, and `private static`. For this reason, option A is correct, and option D is incorrect. Option B is incorrect, as classes are never considered functional interfaces. A functional interface contains exactly one abstract method, although methods that have matching signatures as public methods in `java.lang.Object` do not count toward the single method test. For these reasons, option C is correct. Finally, option E is incorrect. While a functional interface can be marked with the `@FunctionalInterface` annotation, it is not required.

Türkçe: A, C. Functional interface; `default`, `private`, `static` ve `private static` dahil istediği sayıda non-abstract method içerebilir. Bu nedenle A seçeneği doğru, D seçeneği yanlıştır. Class'lar hiçbir zaman functional interface sayılmadığından B seçeneği yanlıştır. Functional interface tam olarak bir abstract method içerir; ancak `java.lang.Object`teki public method'larla eşleşen signature'lara sahip method'lar single method testine katılmaz. Bu nedenlerle C seçeneği doğrudur. Son olarak E seçeneği yanlıştır. Functional interface `@FunctionalInterface` annotation'ıyla işaretlenebilse de bu zorunlu değildir.

Official Answer 4

English: A, F. Option B is incorrect because it does not use the `return` keyword. Options C, D, and E are incorrect because the variable `e` is already in use from the lambda and cannot be redefined. Additionally, option C is missing the `return` keyword, and option E is missing the semicolon. Therefore, options A and F are correct.

Türkçe: A, F. B seçeneği `return` keyword'ünü kullanmadığı için yanlıştır. C, D ve E seçenekleri yanlıştır; çünkü `e` variable'ı lambda tarafından zaten kullanılmaktadır ve yeniden tanımlanamaz. Ayrıca C seçeneğinde `return` keyword'ü, E seçeneğinde ise semicolon eksiktir. Bu nedenle A ve F seçenekleri doğrudur.

Official Answer 5

English: A, C, E. Java includes support for three primitive streams, along with numerous functional interfaces to go with them: `int`, `double`, and `long`. For this reason, options C and E are correct. Additionally, there is a `BooleanSupplier` functional interface, making option A correct. Java does not include primitive streams or related functional interfaces for other numeric data types, making options B and D incorrect. Option F is incorrect because `String` is not a primitive but an object. Only primitives have custom suppliers.

Türkçe: A, C, E. Java; bunlarla birlikte kullanılan çok sayıda functional interface'in yanı sıra üç primitive stream'i destekler: `int`, `double` ve `long`. Bu nedenle C ve E seçenekleri doğrudur. Ayrıca `BooleanSupplier` functional interface'i bulunduğundan A seçeneği de doğrudur. Java, diğer numeric data type'lar için primitive stream veya bunlarla ilişkili functional interface sağlamaz; bu nedenle B ve D seçenekleri yanlıştır. `String` primitive değil object olduğundan F seçeneği yanlıştır. Yalnızca primitive'lerin custom supplier'ları vardır.

Official Answer 6

English: A, C. `Predicate<String>` takes a parameter list of one parameter using the specified type. Options E and F are incorrect because they specify the wrong type. Options B and D are incorrect because they use the wrong syntax for the arrow operator. This leaves us with options A and C as the answers.

Türkçe: A, C. `Predicate<String>`, belirtilen type'ı kullanan tek parameter'lı bir parameter list alır. E ve F seçenekleri yanlış type'ı belirttikleri için yanlıştır. B ve D seçenekleri arrow operator için yanlış syntax kullandığından yanlıştır. Böylece cevap olarak A ve C seçenekleri kalır.

Official Answer 7

English: E. While there appears to have been a variable name shortage when this code was written, it does compile. Lambda variables and method names are allowed to be the same. The `x` lambda parameter is scoped to within each lambda, so it is allowed to be reused. The type is inferred by the method it calls. The first lambda maps `x` to a `String` and the second to a `Boolean`. Therefore, option E is correct.

Türkçe: E. Bu kod yazılırken variable adı kıtlığı yaşanmış gibi görünse de kod derlenir. Lambda variable'ları ile method adlarının aynı olmasına izin verilir. Lambda parameter `x`'in scope'u her bir lambda'nın içidir; bu nedenle yeniden kullanılabilir. Type, çağrılan method tarafından infer edilir. İlk lambda'daki `x`, `String`; ikincisindeki ise `Boolean` type'ına eşlenir. Dolayısıyla E seçeneği doğrudur.

Official Answer 8

English: E. The question starts with a `UnaryOperator<Integer>`, which takes one parameter and returns a value of the same type. Therefore, option E is correct, as `UnaryOperator` extends `Function`. Notice that other options don't even compile because they have the wrong number of generic types for the functional interface provided. You should know that a `BiFunction<T,U,R>` takes three generic arguments, a `BinaryOperator<T>` takes one generic argument, and a `Function<T,R>` takes two generic arguments.

Türkçe: E. Soru, bir parameter alan ve aynı type'ta value döndüren `UnaryOperator<Integer>` ile başlar. `UnaryOperator`, `Function`ı extend ettiğinden E seçeneği doğrudur. Diğer seçeneklerin, verilen functional interface için yanlış sayıda generic type kullandıkları için derlenmediğine dikkat edin.

`BiFunction<T,U,R>`'nin üç, `BinaryOperator<T>`'nin bir, `Function<T,R>`'nin ise iki generic argument aldığını bilmelisiniz.

Official Answer 9

English: A, F. Option A is correct and option B is incorrect because a `Supplier` returns a value while a `Consumer` takes one and acts on it. Option C is tricky. `IntSupplier` does return an `int`. However, the option asks about `IntegerSupplier`, which doesn't exist. Option D is incorrect because a `Predicate` returns a `boolean`. It does have a method named `test()`, making option F correct. Finally, option E is incorrect because `Function` has an `apply()` method.

Türkçe: A, F. `Supplier` value döndürürken `Consumer` bir value alıp onun üzerinde işlem yaptığından A seçeneği doğru, B seçeneği yanlıştır. C seçeneği yanıltıcıdır. `IntSupplier` gerçekten `int` döndürür; ancak seçenek, var olmayan `IntegerSupplier`'ı sorar. `Predicate` `boolean` döndürdüğü için D seçeneği yanlıştır. `Predicate`'in `test()` adlı method'u bulunduğu için F seçeneği doğrudur. Son olarak `Function`'in `apply()` method'u bulunduğu için E seçeneği yanlıştır.

Official Answer 10

English: A, B, C. Since the scope of `start` and `c` is within the lambda, the variables can be declared or updated after it without issue, making options A, B, and C correct. Option D is incorrect because setting `end` prevents it from being effectively final.

Türkçe: A, B, C. `start` ve `c` nin scope'u lambda'nın içinde olduğu için variable'lar lambda'dan sonra sorunsuz biçimde bildirilebilir veya update edilebilir; dolayısıyla A, B ve C seçenekleri doğrudur. `ende` değer atamak onun effectively final olmasını engellediğinden D seçeneği yanlıştır.

Official Answer 11

English: D. The code does not compile because the lambdas are assigned to `var`. The compiler does not have enough information to determine they are of type `Predicate<String>`. Therefore, option D is correct.

Türkçe: D. Lambda'lar `vara` atıldığı için kod derlenmez. Compiler, bunların `Predicate<String>` type'ında olduğunu belirlemek için yeterli bilgiye sahip değildir. Dolayısıyla D seçeneği doğrudur.

Official Answer 12

English: A. The `a.compose(b)` method calls the `Function` parameter `b` before the reference `Function` variable `a`. In this case, that means that we multiply by 3 before adding 4. This gives a result of 7, making option A correct.

Türkçe: A. `a.compose(b)` method'u, reference `Function` variable `adan` önce `Function` parameter `byi` çağırır. Bu sorudaki adlarla kaynak önce `t`yi, sonra `s`yi çalıştırmayı anlatmaktadır. Dolayısıyla önce 3 ile çarpıp sonra 4 ekleriz. Sonuç 7 olur ve A seçeneği doğrudur.

Editör notu: Resmî açıklama `a.compose(b)` adlarını kullanır; Question 12'de aynı roller sırasıyla `s.compose(t)` olarak adlandırılmıştır.

Official Answer 13

English: E. Lambdas are only allowed to reference final or effectively final variables. You can tell the variable `j` is effectively final because adding a `final` keyword before it wouldn't introduce a compiler error. Each time the `else` statement is executed, the variable is redeclared and goes out of scope. Therefore, it is not reassigned. Similarly, `length` is effectively final. There are no compiler errors, and option E is correct.

Türkçe: E. Lambda'ların yalnızca `final` veya effectively final variable'lara reference vermesine izin verilir. Önüne `final` keyword'ü eklendiğinde compiler error oluşmayacağından `j` variable'ının effectively final olduğunu anlayabilirsiniz. `else` statement her çalıştığında variable yeniden bildirilir ve scope'tan çıkar; dolayısıyla reassignment yapılmaz. Benzer biçimde `length` de effectively finaldır. Compiler error yoktur ve E seçeneği doğrudur.

Official Answer 14

English: B, D. Option B is a valid functional interface, one that could be assigned to a `Consumer<Camel>` reference. Notice that the `final` modifier is permitted on variables in the parameter list. Option D is correct, as the exception is being returned as an object and not thrown. This would be compatible with a `BiFunction` that included `RuntimeException` as its return type.

Türkçe: B, D. Kaynağın ifadesiyle B seçeneği, bir `Consumer<Camel>` reference'a atanabilecek geçerli bir functional interface kullanımıdır. Parameter list'teki variable'larda `final` modifier'a izin verildiğine dikkat edin. Exception fırlatılmayıp object olarak döndürüldüğü için D seçeneği de doğrudur. Bu ifade, return type'i `RuntimeException` olan bir `BiFunction` ile compatible olabilir.

Editör notu: B seçeneğinin kendisi bir functional interface declaration değil, `Consumer<Camel>` ile compatible olabilecek geçerli bir lambda expression'dır; resmî açıklamadaki terim bu anlamda yorumlanmalıdır.

English: Options A and G are incorrect because they mix format types for the parameters. Option C is invalid because the variable `b` is used twice. Option E is incorrect, as a `return` statement is permitted only inside braces (`{}`). Option F is incorrect because the variable declaration requires a semicolon (`;`) after it.

Türkçe: A ve G seçenekleri parameter'lar için farklı format type'larını karıştırdıkları için yanlıştır. C seçeneği `b` variable'ını iki kez kullandığından geçersizdir. `return` statement'a yalnızca braces (`{}`) içinde izin verildiğinden E seçeneği yanlıştır. Variable declaration sonrasında semicolon (`;`) gerektiği için F seçeneği yanlıştır.

Official Answer 15

English: A, F. Option A is a valid lambda expression. While `main()` is a static method, it can access `age` since it is using a reference to an instance of `Hyena`, which is effectively final in this method. Since `var` is not a reserved word, it may be used for variable names. Option F is also correct, with the lambda variable being a reference to a `Hyena` object. The variable is processed using deferred execution in the `testLaugh()` method.

Türkçe: A, F. A seçeneği geçerli bir lambda expression'dır. `main()` static method olmasına rağmen bu method'da effectively final olan bir `Hyena` instance reference'ı kullandığı için `age` erişilebilir. `var` reserved word olmadığından variable adı olarak kullanılabilir. Lambda variable'ı bir `Hyena` object'ine reference verdiğinden F seçeneği de doğrudur. Variable, `testLaugh()` method'unda deferred execution ile işlenir.

English: Options B and E are incorrect; since the local variable `age` is not effectively final, this would lead to a compilation error. Option C would also cause a compilation error, since the expression uses the variable name `p`, which is already declared within the method. Finally, option D is incorrect, as this is not even a lambda expression.

Türkçe: Local variable `age` effectively final olmadığından B ve E seçenekleri compilation error'a yol açar ve yanlıştır. Expression, method içinde zaten bildirilmiş olan `p` variable adını kullandığından C seçeneği de compilation error oluşturur. Son olarak D seçeneği bir lambda expression bile olmadığından yanlıştır.

Official Answer 16

English: C. Lambdas are not allowed to redeclare local variables, making options A and B incorrect. Option D is incorrect because setting `end` prevents it from being effectively final. Lambdas are only allowed to reference final or effectively final variables. Option C compiles since `chars` is not used.

Türkçe: C. Lambda'ların local variable'ları yeniden bildirmesine izin verilmediğinden A ve B seçenekleri yanlıştır. `ende` değer atamak onun effectively final olmasını engellediği için D seçeneği yanlıştır. Lambda'lar yalnızca `final` veya effectively final variable'lara reference verebilir. `chars` kullanılmadığından C seçeneği derlenir.

Official Answer 17

English: C. Line 8 uses braces around the body. This means the `return` keyword and semicolon are required. Since the code doesn't compile, option C is the answer.

Türkçe: C. Satır 8 body'nin çevresinde braces kullanır. Bu nedenle `return` keyword'ü ve semicolon zorunludur. Kod derlenmediğinden cevap C seçeneğidir.

Official Answer 18

English: B, F, G. We can eliminate four choices right away. Options A and C are there to mislead you; these interfaces don't exist. Option D is incorrect because a `BiFunction<T,U,R>` takes three generic arguments, not two. Option E is incorrect because none of the examples returns a `boolean`.

Türkçe: B, F, G. Dört seçeneği hemen eleyebiliriz. A ve C seçenekleri sizi yanıltmak için vardır; bu interface'ler mevcut değildir. D seçeneği yanlıştır; çünkü `BiFunction<T,U,R>` iki değil üç generic argument alır. Örneklerin hiçbirisi `boolean` döndürmediğinden E seçeneği de yanlıştır.

English: The declaration on line 6 doesn't take any parameters, and it returns a `String`, so a `Supplier<String>` can fill in the blank, making option F correct. The declaration on line 7 requires you to recognize that `Consumer` and `Function`, along with their binary equivalents, have an `andThen()` method. This makes option B correct. Finally, line 8 takes a single parameter, and it returns the same type, which is a `UnaryOperator`. Since the types are the same, only one generic parameter is needed, making option G correct.

Türkçe: Satır 6'daki declaration parameter almaz ve `String` döndürür; dolayısıyla boşluğu `Supplier<String>` doldurabilir ve F seçeneği doğrudur. Satır 7'deki declaration, `Consumer` ve `Function`ın binary equivalent'larıyla birlikte bir `andThen()` method'una sahip olduğunu fark etmenizi gerektirir. Bu, B seçeneğini doğru yapar. Son olarak satır 8 tek parameter alır ve aynı type'ı döndürür; bu bir `UnaryOperator`dir. Type'lar aynı olduğundan yalnızca bir generic parameter gerekir ve G seçeneği doğrudur.

Official Answer 19

English: F. While there is a lot in this question trying to confuse you, note that there are no options about the code not compiling. This allows you to focus on the lambdas and method references. Option A is incorrect because a `Consumer` requires one parameter. Options B and C are close. The syntax for a lambda is correct. However, `s` is already defined as a local variable, and therefore the lambda can't redefine it. Options D and E use incorrect syntax for a method reference. Option F is correct.

Türkçe: F. Bu soruda sizi şaşırtmaya çalışan çok sayıda ayrıntı olsa da seçenekler arasında kodun derlenmemesiyle ilgili bir ifade bulunmadığına dikkat edin. Böylece lambda ve method reference'lara odaklanabilirsiniz. `Consumer` bir parameter gerektirdiğinden A seçeneği yanlıştır. B ve C seçenekleri doğruya yakındır; lambda syntax'ı doğrudur. Ancak `s` zaten local variable olarak tanımlanmıştır ve bu nedenle lambda onu yeniden tanımlayamaz. D ve E seçenekleri method reference için yanlış syntax kullanır. F seçeneği doğrudur.

Official Answer 20

English: E. Option A does not compile because the second statement within the block is missing a semicolon (;) at the end. Option B is an invalid lambda expression because `t` is defined twice: in the parameter list and within the lambda expression. Options C and D are both missing a `return` statement and semicolon. Options E and F are both valid lambda expressions, although only option E matches the behavior of the `Sloth` class. In particular, option F only prints `Sleep:`, not `Sleep: 10.0`.

Türkçe: E. Block içindeki ikinci statement'ın sonunda semicolon (;) eksik olduğundan A seçeneği derlenmez. `t`, parameter list'te ve lambda expression içinde olmak üzere iki kez tanımlandığından B seçeneği geçersiz bir lambda expression'dır. C ve D seçeneklerinin ikisinde de `return` statement ve semicolon eksiktir. E ve F seçeneklerinin ikisi de geçerli lambda expression'lardır; ancak yalnızca E seçeneği `Sloth` class'ının davranışıyla eşleşir. Özellikle F seçeneği `Sleep: 10.0` değil, yalnızca `Sleep:` yazdırır.

Official Answer 21

English: A, E, F. A valid functional interface is one that contains a single abstract method, excluding any public methods that are already defined in the `java.lang.Object` class. `Transport` and `Boat` are valid functional interfaces, as they each contain a single abstract method: `go()` and `hashCode(String)`, respectively. This gives us options A and E. Since the other methods are part of `Object`, they do not count as abstract methods. `Train` is also a functional interface since it extends `Transport` and does not define any additional abstract methods. This adds option F as the final correct answer.

Türkçe: A, E, F. Geçerli functional interface, `java.lang.Object` class'ında zaten tanımlanmış public method'lar hariç tutulduğunda tek bir abstract method içerir. `Transport` ve `Boat`, sırasıyla `go()` ve `hashCode(String)` olmak üzere tek bir abstract method içerdiklerinden geçerli functional interface'lerdir. Böylece A ve E seçeneklerini elde ederiz. Diğer method'lar `Object`'in parçası olduğundan abstract method olarak sayılmaz. `Train`, `Transport`'ı extend edip ek abstract method tanımlamadığı için o da functional interface'tir. Böylece son doğru cevap olarak F seçeneği eklenir.

English: `Car` is not a functional interface because it is an abstract class. `Locomotive` is not a functional interface because it includes two abstract methods, one of which is inherited. Finally, `Spaceship` is not a valid interface, let alone a functional interface, because a default method must provide a body. A quick way to test whether an interface is a functional interface is to apply the `@FunctionalInterface` annotation and check if the code still compiles.

Türkçe: `Car` abstract class olduğu için functional interface değildir. `Locomotive`, biri inherited olmak üzere iki abstract method içerdiğinden functional interface değildir. Son olarak default method bir body sağlamak zorunda olduğundan `Spaceship` geçerli bir interface bile değildir; functional interface hiç değildir. Bir interface'in functional interface olup olmadığını hızlıca test etmenin yolu, `@FunctionalInterface` annotation'ını uygulayıp kodun hâlâ derlenip derlenmediğini kontrol etmektir.